
PROBLEM SOLVING AGENTS: SEARCHING

INSTRUCTOR: MOAYAD ALNAMMI

malnammi.github.io

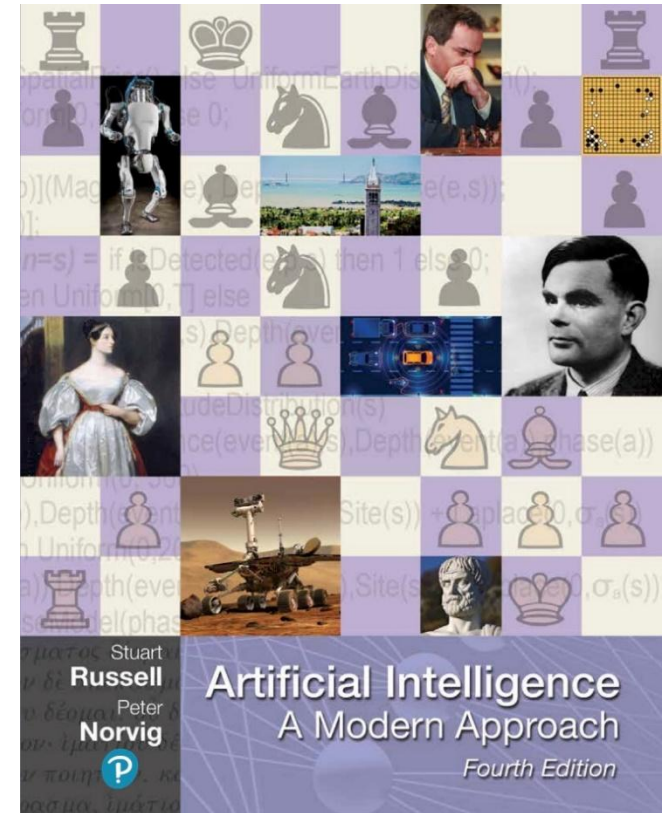
Notice

- Slides prepared by Moayad Alnammi – malnammi.github.io
- Some figures adapted from CS188: Introduction to Artificial Intelligence, UC Berkeley (Dan Klein, Pieter Abbeel) — ai.berkeley.edu
- Some figures are obtained from the [AIMA Book](#)
- Some content adapted from Dr. ElSayed El-Alfy

This deck is published at malnammi.github.io under CC BY-NC-SA 4.0. Reuse and adaptation require attribution to the author and the sources above; commercial use is not permitted; adaptations must carry the same license.

Resources & Reading

- Textbook website: <http://aima.eecs.berkeley.edu/>
- Read Chapter 3.1-3.4 in AIMI 4/E



Outline

- Problem-Solving Agents – Searching
- Formulating Problems
- Example Problems
- Searching for Solutions – Search Algorithms
- Evaluating Search Algorithms
- Best-first Search Algorithm (Template Algorithm)
- Uninformed Search Algorithms:
 - Breadth-first search (BFS)
 - Uniform-cost search (UCS)
 - Depth-first search (DFS)
 - Depth-limited search
 - Iterative-deepening search (ID)
 - Bidirectional search
- Summary

What are problem-solving agents?

Problem-solving agents = searching agents [1]

- For problems with a goal, the agent can search for a *sequence of actions* that would form a *path* to the goal.
- In this lecture we will discuss various *search* algorithms for finding a *solution* (*sequence of actions*) to a goal.
- We make the following simplifying assumptions about the *environment*:
 - Single-agent
 - Static: no changes of environment while solving the problem (computing solution).
 - Observable: states have complete information.
 - Discrete: finite set of states and actions.
 - Deterministic: next state is determined by the current state and action taken.
 - Known: the dynamics/physics of the environment are known. This means that the outcome of actions are specified completely.
 - A *problem instance* just involves getting from an initial state to a goal state. So, it is sequential within a given problem instance (current action can affect future actions). But you can also say it is episodic between two problem instances (path taken for one instance is independent of path taken for another instance).

Problem-solving agents = searching agents [2]

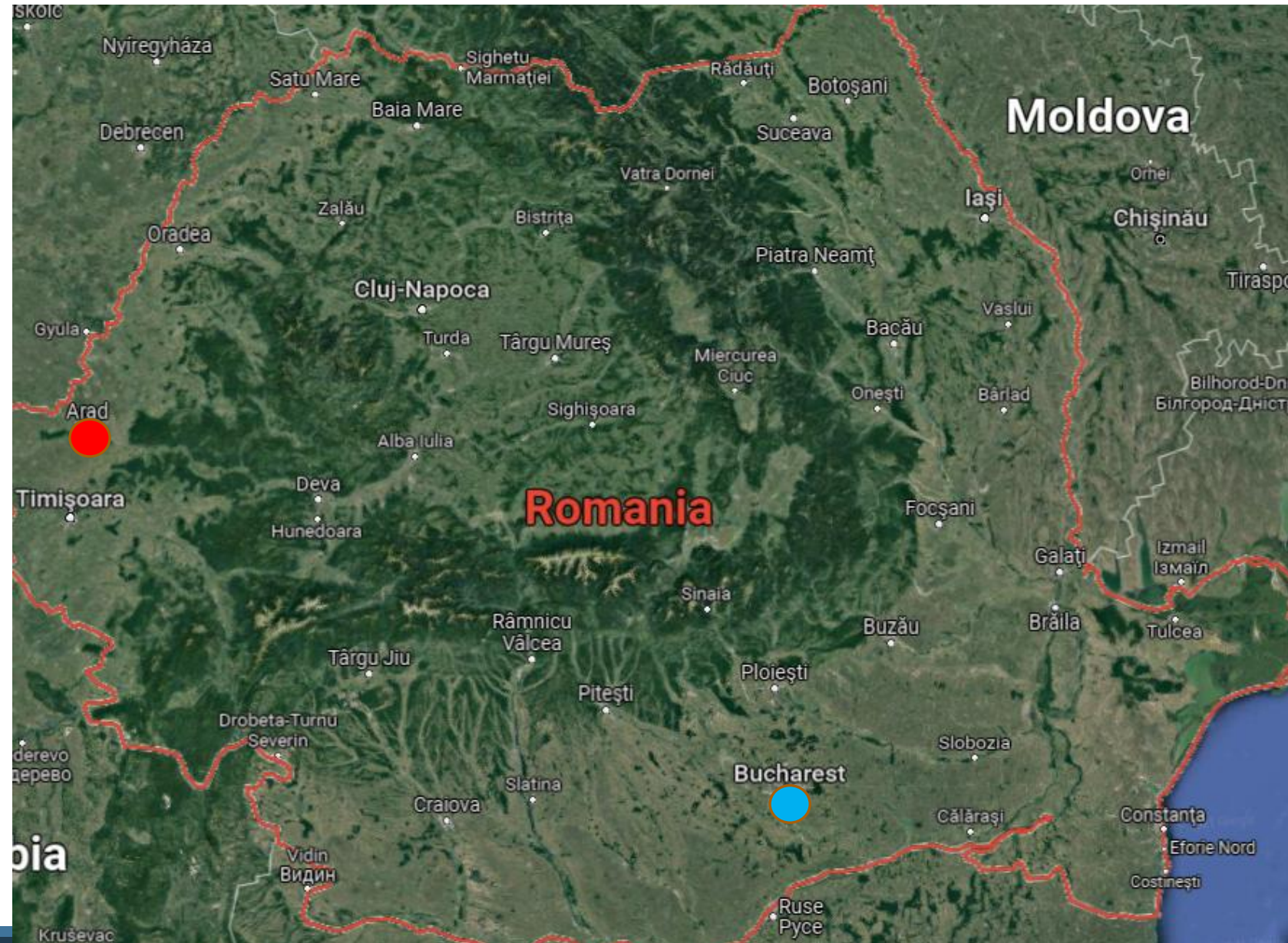
- Search algorithm takes a **problem instance** as input and generates an **action sequence** as output (**solution**).
- Design of searching agents:
 - **Goal formulation:** need to define the goal of the given problem.
 - **Problem formulation (modelling):** need to develop a **representation** of the states and actions. This is the agent's **abstract model** of the problem. **[Important learning outcome for this course]**
 - **Search:** given the abstract model and the goal, the agent **computationally searches (simulation)** for a sequence of actions that will reach the goal (i.e. find a solution).
 - **Execution:** the agent now executes the solving actions in the real-world.
- Typically, we want a solution with desirable properties like minimal cost, safe, avoids risk, etc. We can adopt an evaluation function to judge how good a solution is.

Formulating search problems

- We saw in the previous slide that we need to define the goal and formulate the problem before searching. So how can we formulate a search problem?
- A search problem is defined as follows:
 - **State representation**: variables representing changing properties of the problem.
 - **State space**: **set** of all possible states in the search problem.
 - **Initial state** s_0 . The initial state as determined by the problem instance.
 - **Actions-function** $A(s)$: The set of *allowable* actions in state s .
 - **Transition model** $Result(s,a)$. Returns the state that would result from executing action a in state s .
 - **Goal test** $G(s)$. Function to check if the state s is the **goal state**.
 - **Action cost** $c(s,a,s')$. Returns the numeric cost of executing action a in state s to reach state s' .
- These components can be *represented visually* as a graph called the **state space graph**: **nodes** are states and **directed edges** are actions.
- **Path**: is a sequence of states in the state space graph connected by a sequence of actions.
- **Solution**: a path from the initial state to the goal state.
- **Optimal solution**: a solution that has **minimal path cost**.

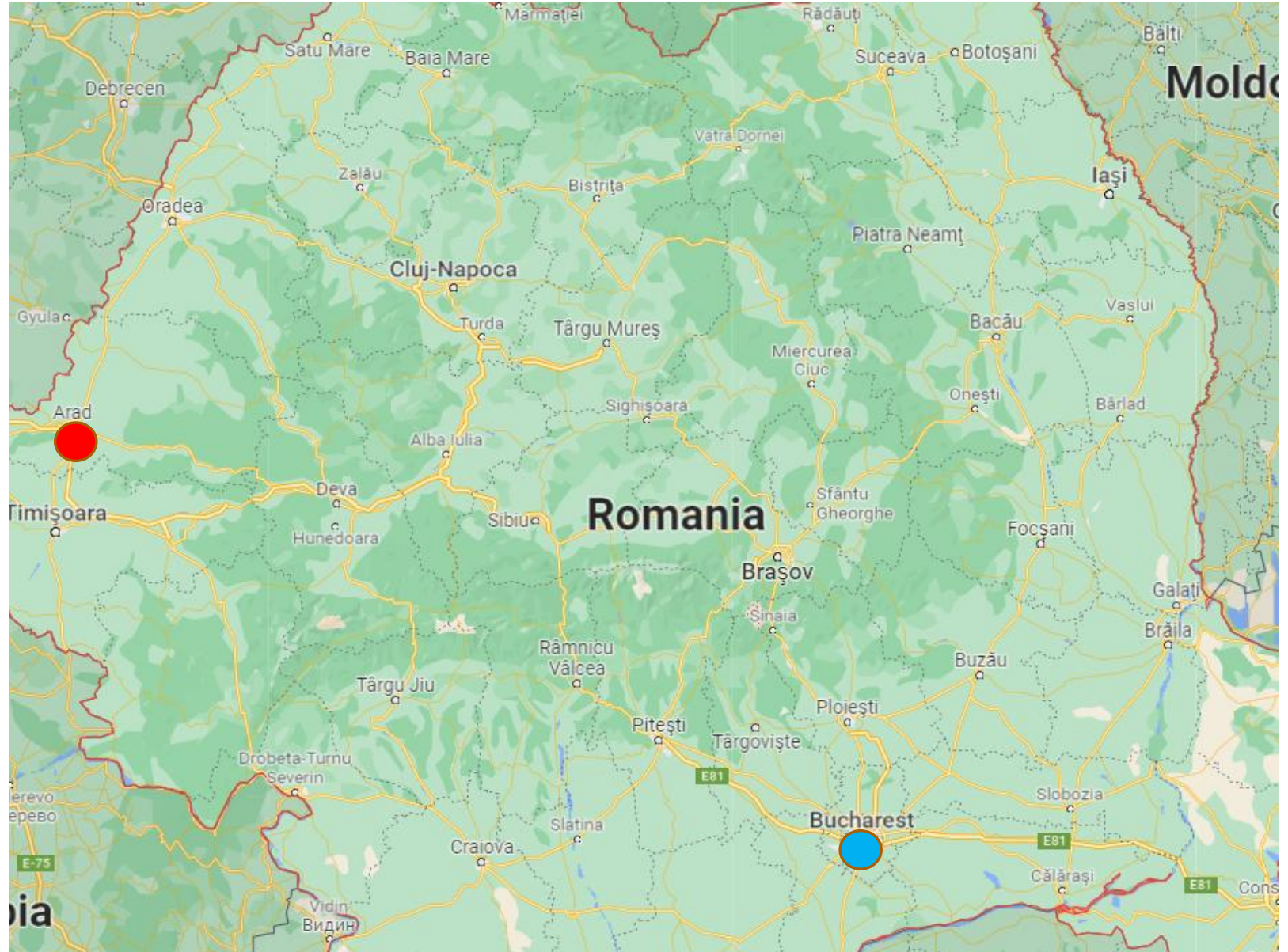
Example: Travelling in Romania [1]

- We want to travel from city Arad to city Bucharest.
- Modelling Romania can be very complex with roads, traffic, tolls, etc.
- But here, we **simplify** the **model** by just considering how to get from one major city to another along major roads.



Example: Travelling in Romania [2]

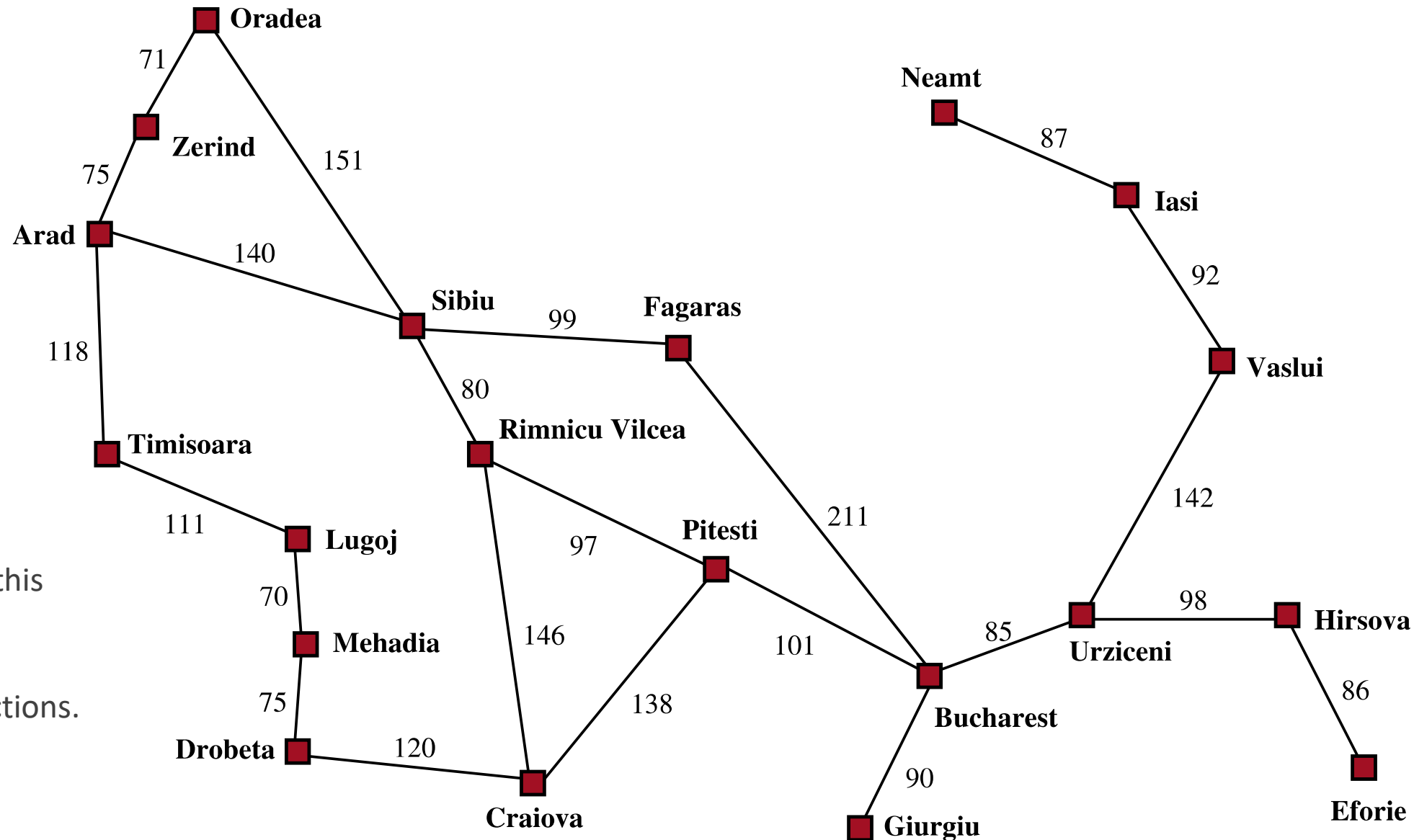
- Here, you can see Google map's **modelling** of Romania.
- Major cities, major roads, minor cities, minor roads.
- Different level of abstractions:
 - zoom out -> more abstraction of cities and major roads.
 - zoom in -> more details of minor roads and city blocks.
- In this class we will use even simpler modelling of this travelling problem for sake of learning.



Example: Travelling in Romania [3]

- We want to travel from city Arad to city Bucharest. Modelling Romania can be very complex with roads, traffic, tolls, etc. But here, we **simplify** the **model** by just considering how to get from one major city to another. Connection between states (major cities) will be simplified to the main road between them.
- Available info: agent knows what city he is currently in, agent knows roads between cities.
- **state representation**: a state will be the city the agent is currently in. In other words, *state* is a single variable for the city (could use string or int).
- **state space**: Total of 20 cities, so state-space has 20 unique states.
- **initial state**: $s_0 = "Arad"$
- **goal test** $G(s)$: being in Bucharest. Logical syntax it is the condition $s == "Bucharest"$
- **Actions** $A(s)$: since road info available, we can code this to return list of neighbor cities. (python dictionary)
- **Transition model** $Result(s, a)$: return city s' as determined by taking road a in city s .
- **Action cost** $c(s, a, s')$: distance or cost of going from city s to s' . (python dictionary)
- **Solution**: a path starting in Arad and ending in Bucharest.

Example: Travelling in Romania [4] – state space graph



Nodes are states (city for this specific problem)

Edges here represent two directed edges, i.e. two actions.

Costs are shown

Implementing a search problem

- We'll use the following structure to implement a search problem:
 - Constructor `__init__`: this will take in arguments to initialize the problem instance (like locations, dimensions, etc.). Must specify `self.initial_state`
 - `actions(state)`: returns list of *allowable* actions in *state*.
 - `result(state, act)`. Returns the state that would result from executing action *act* in state *state*.
 - `is_goal(s)`. Function to check if the state *s* is the goal state.
 - `cost(state, action, next_state)`. Returns the numeric cost of executing *action* in *state* to reach *next_state*.
- To implement our own search problem, we just need to make sure to define these functions. Why? Because the search algorithms will call them as they search (we'll see later).

```
1 class SearchProblem(object):
2
3     def __init__(self, initial_state=None):
4         self.initial_state = initial_state
5
6     def actions(self, state):
7         raise NotImplementedError
8
9     def result(self, state, action):
10        raise NotImplementedError
11
12    def cost(self, state, action, state2):
13        return 1
14
15    def is_goal(self, state):
16        raise NotImplementedError
17
18    def heuristic(self, state):
19        return 0
20
```

Example: Vacuum Cleaner World [1] – state space graph

- For 2 grids, we have 2 possible locations for the agent, each grid can be clean or dirty (2 choices per grid).
- $s = [loc, cleanA, cleanB]$
- This gives $2 * 2^2 = 8$ states.
- What about if we had n grids?
- **Notice** that a **state** in this problem consists of: agent grid-location, and grid clean-status. In Romania problem, a state is just the city the agent is in.
- We are using as few variables as possible to solve the problem (for efficiency).

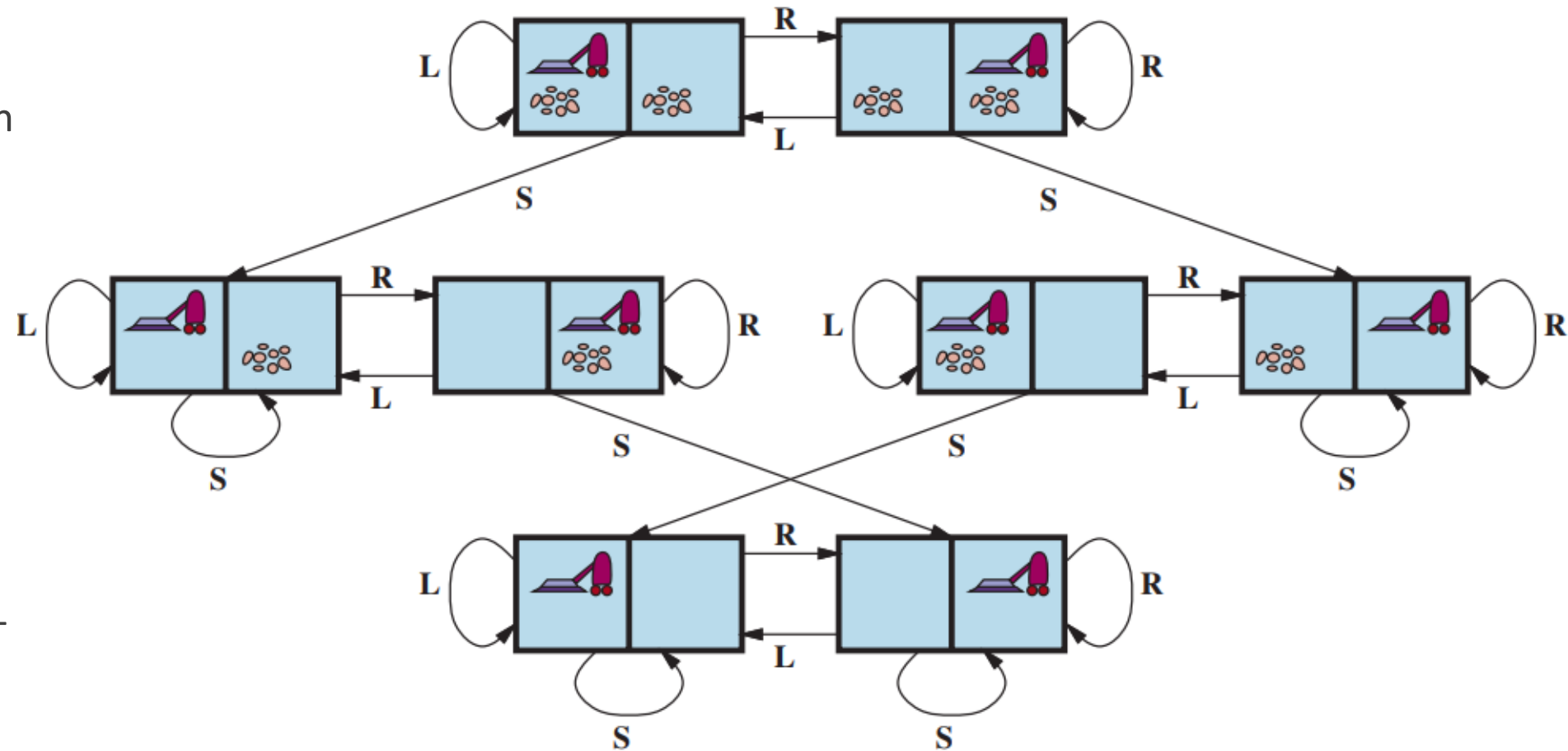


Figure 3.2 The state-space graph for the two-cell vacuum world. There are 8 states and three actions for each state: L = *Left*, R = *Right*, S = *Suck*.

Example: Sliding block puzzles

- State representation?
 - $s = 3 \times 3$ int matrix (other data-structures possible)
- State-space?
 - Any configuration of the tiles. How many possible?
- Actions?
 - Clever idea to simplify coding: move the *blank* tile to an adjacent location. Left, right, up, down.
- Initial state?
 - Some configuration is set as initial.
- Goal test?
 - Check if current configuration (state) is the desired one. (hard-code check)
- Action cost?
 - 1 per move.
- Optimal solution?
 - Get from initial state to goal state in as few moves as possible.
- Interesting note: in this problem, some initial positions are unsolvable (see [link](#))

7	2	4
5		6
8	3	1

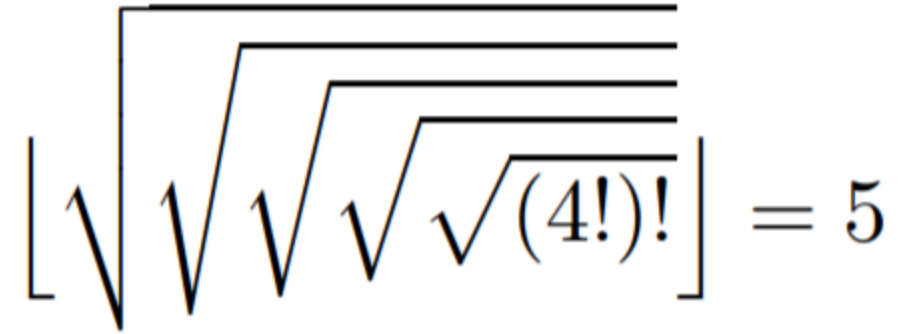
Start State

	1	2
3	4	5
6	7	8

Goal State

Example: Infinite state space

- State representation?
 - $s \in \mathbb{Z}^+$ pos int
- State-space?
 - infinite state space
- Actions?
 - Apply square root, floor, or factorial.
- Initial state?
 - $s_0 = 4$
- Goal test?
 - $s == goalint$
- Action cost?
 - 1 per operation
- Optimal solution?
 - Get from initial state to goal state in as few costs as possible.



The diagram illustrates a sequence of operations on the number 4. It shows a series of nested square root symbols. The innermost expression is $(4!)!$. This is followed by a square root, then another square root, and so on, with the nesting continuing to the left. The entire expression is enclosed in large square brackets, and the result is shown as $= 5$. The visual representation of the nested square roots is achieved by drawing horizontal lines that extend from the top of each square root symbol to the top of the next one to its left, creating a staircase-like effect.

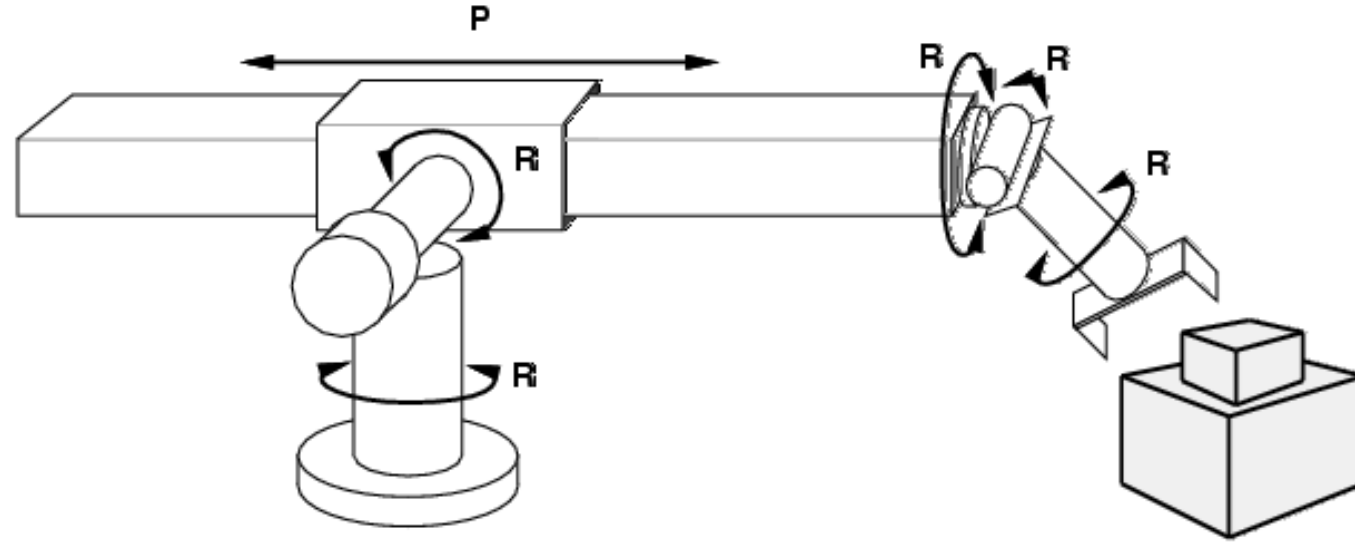
$$\left[\sqrt{\sqrt{\sqrt{\sqrt{\sqrt{(4!)!}}}}} \right] = 5$$

Donald Knuth conjectured that, starting with the number 4, a sequence of factorial, square root, and floor operations will reach **any** desired positive integer.

Real-world search problems

- Tend to be more difficult but whose solutions are what people actually care about.
- Large-scale Route-finding: defined in terms of specified locations and transitions along links between them. Its applications are (Google map, routing in computer networks, automated travel advisory system)
- Robot navigation: similar to the route-finding problem, but there is infinite set of possible actions and states (less abstraction).
- VLSI layout: positioning (transistors, resistors, capacitors, etc) and connections among a million gates in the silicon chip is complicated and finding optimal way to place components on a printed circuit board such that:
 - Minimize surface area
 - Minimize number of signal layers
 - Minimize number of connections from one layer to another
 - Minimize length of some signal lines (e.g., clock line)
 - Distribute heat throughout board

Example: Robot Assembly



- States: real-valued coordinates of robot (angles, positions) and object parts.
- Initial state: any configuration of robot and object parts.
- Actions: continuous motion of robot joints.
- Goal test: object assembled.
- Path Cost: time taken or number of actions.

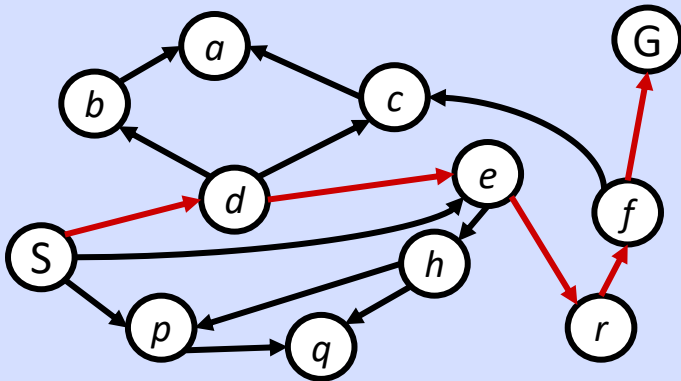
Search Algorithms

Search Algorithms – state-space graph vs search tree

- After problem formulation, find a solution. That is, determine action sequence starting from initial state and leading to goal state.
- **Search algorithms**: takes a search problem as input, does a *computational search*, then produces a solution or failure.
- Search algorithms will *computationally* operate using a **search tree**.

State-Space Graph

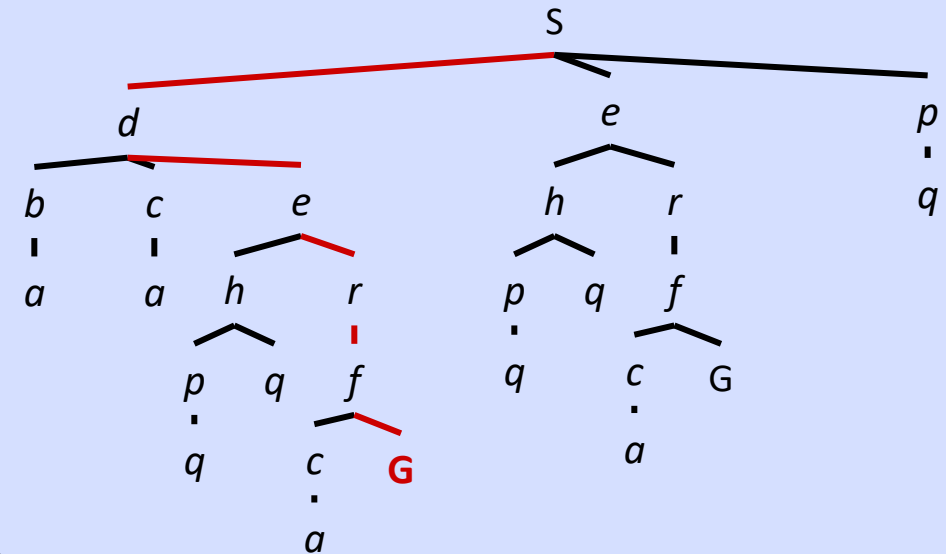
(for some problem)



Each NODE in in the search tree is a PATH in the state space graph.

We construct the tree on demand – and we construct as little as possible.

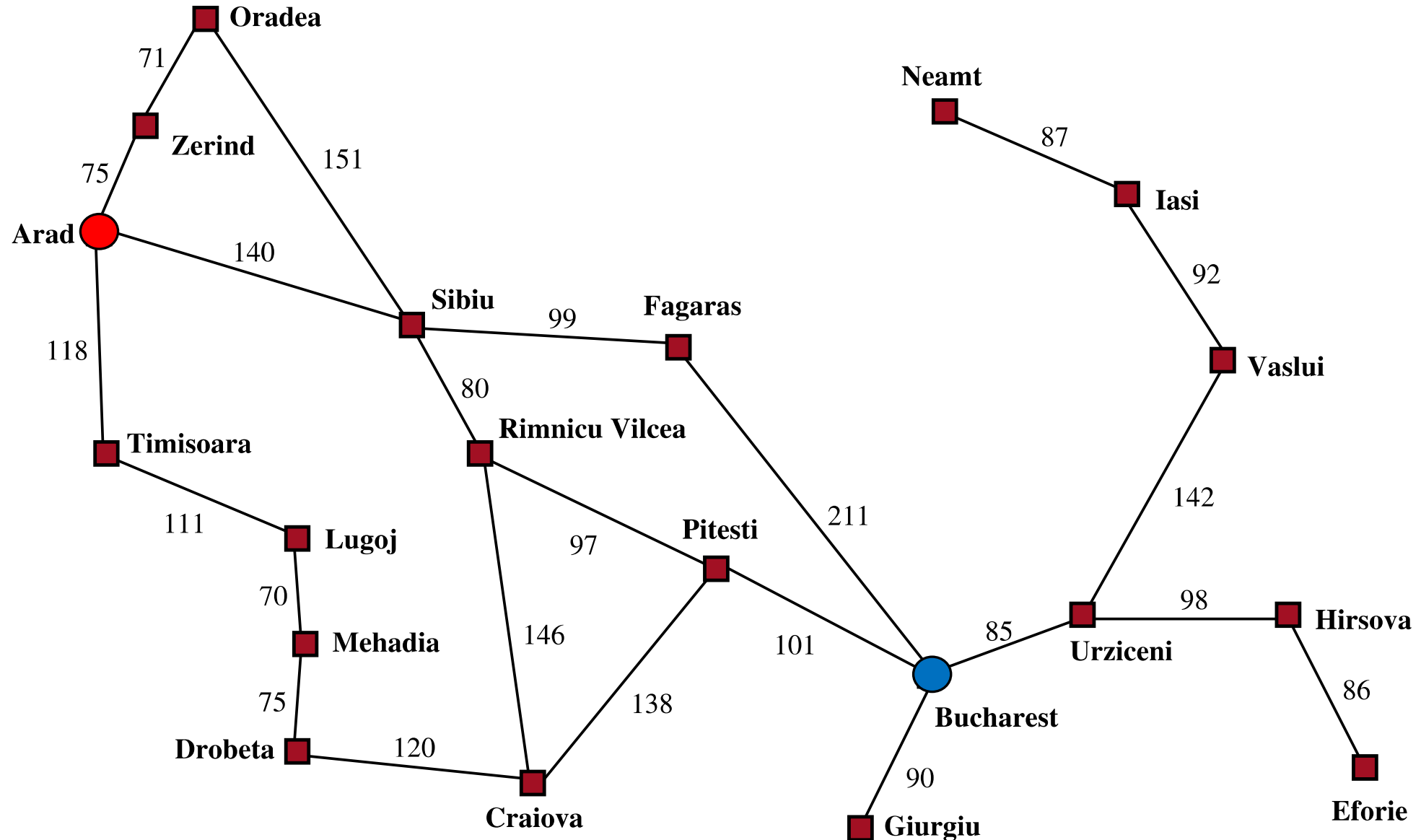
Search Tree



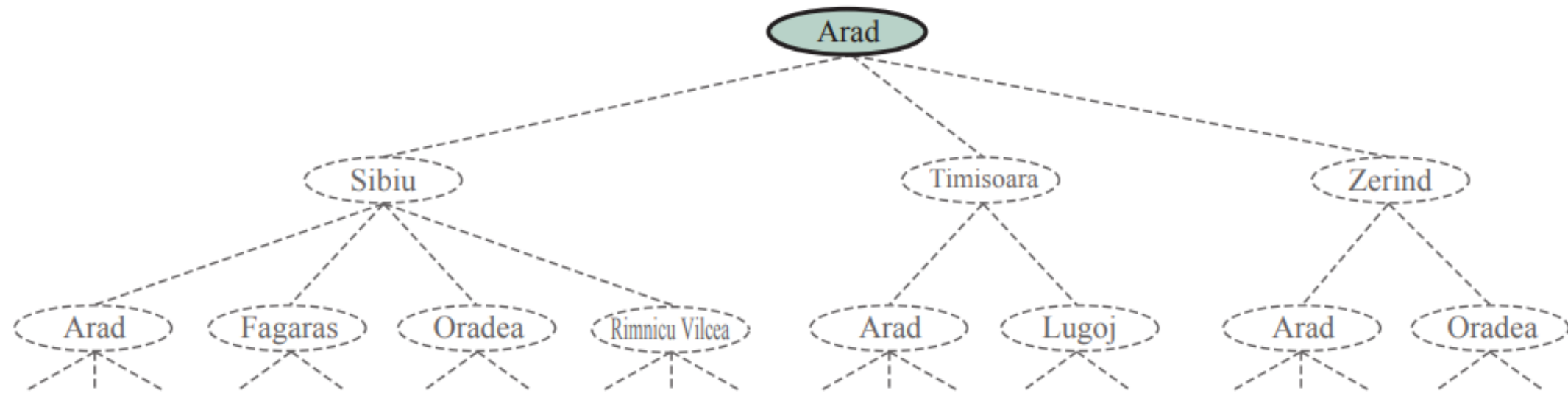
Search Algorithms – state-space graph vs search tree

- Search algorithms will computationally operate using a **search tree**:
 - This is not the same as the state-space graph. The state-space graph gives us the model of the problem (transitions in particular). The search tree is what search algorithms **grow and maintain** to find a path in the state-space graph.
 - **Nodes** in the search tree correspond to some state in the problem state-space. A state can be in more than one node in the search tree.
 - **Branches** in the search tree correspond to actions.
 - Start at the root (initial state), and **expand** the tree by generating successors (next states); test each node: is it a goal state?; stop when reaching a goal.
 - A **path** from a node can be followed back to the root.
 - We can keep track of already visited states and not allow for repetition visits to increase search efficiency. This is called keeping track of **redundant paths** and costs extra memory storage. In this case we refer to the search as a **graph-like** (because our search tree will not have any duplicates along a single path).
 - If we do not keep track of visited states, the search is called **tree-like**. For problems with cyclic paths, some search algorithms can get stuck in these cycles and never terminate.

Example: Travelling in Romania – state space graph

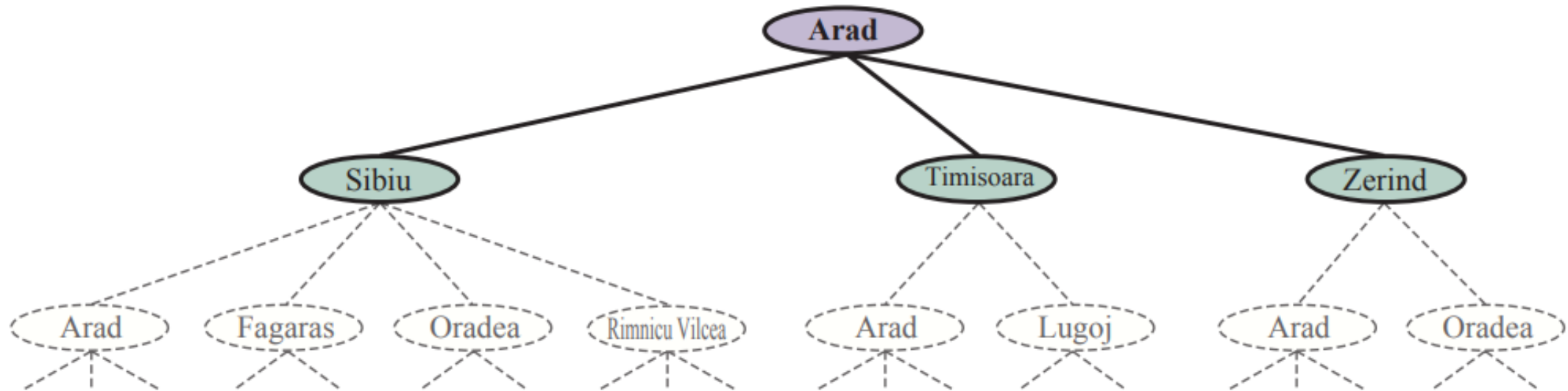


Example: Travelling in Romania – search tree [1]



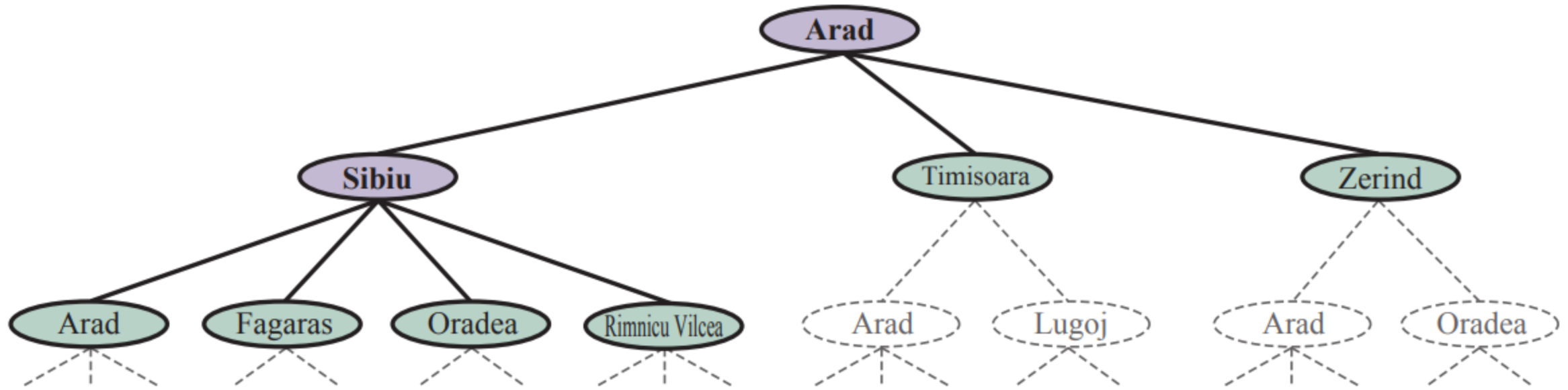
- At the start, we have a node for the initial state s_0 (Arad)
- We **expand** this node by looking at available actions $A(s_0)$ and transition $Result(s_0, a)$.
- **Expanding** a node will **generate** the children/successors of the state.

Example: Travelling in Romania – search tree [2]



- We have three choices for expansion: go to Sibiu, Timisoara, or Zerind.
- Searching: which child node should we expand next?
- Let's say we expand Sibiu next. This will generate 4 children for Sibiu.

Example: Travelling in Romania – search tree [2]

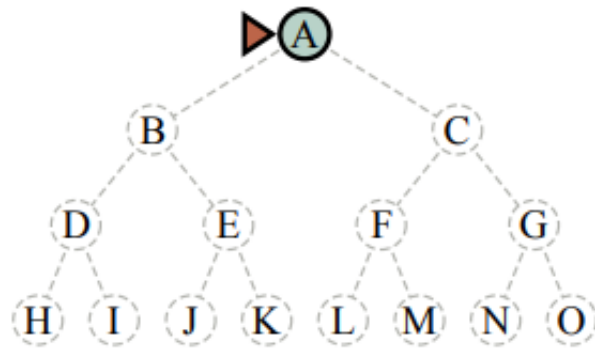


- Let's say we expand Sibiu next. This will generate 4 children for Sibiu.
- **Expanded** nodes are in **purple**.
- **Frontier**: The nodes (**green**) that have been generated but have **not** yet been expanded. These nodes are **candidates** for expansion by the search algorithm.
- **Reached Nodes**: these are nodes that have been expanded or generated (**green** + **purple** nodes). Maintaining this will allow us to keep track of repeated states (redundant/cycle paths).
- In our search implementation, we will maintain two data-structures: **Frontier** and **Reached**

Search Algorithm Template: Best-first Search [1]

- **Frontier:** The nodes (green) that have been generated but have **not** yet been expanded. These nodes are **candidates** for expansion by the search algorithm.
- **Reached Nodes:** these are nodes that have been expanded or generated (green + purple nodes). Maintaining this will allow us to keep track of repeated states (redundant/cycle paths).
- How do we select which node to expand from the frontier next?
- **Best-first search (shown in next slide):** A *template* search algorithm where we define an **evaluation function** $f(n)$ that assigns a value to a node.
- Search strategy of best-first search: select the node from the frontier that has minimum $f(n)$.
- So, we proceed in iterations. On each iteration, we select the node from the frontier with $\min f(n)$, check if it is a goal state (return as solution), otherwise expand this node and add children to frontier.
- We can check for repeated states (graph-like search). So, children are added to frontier if they have not been reached, or if the path to reach this state is better than what we currently know.
- If the frontier is empty, this means we've exhausted all possible paths, and none give us a path to the goal. This means there is no solution (failure).

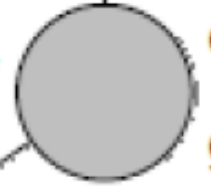
Search Tree Node [1]



State

5	4	
6	1	8
7	3	2

Node



depth = 6

g = 6

state

- Search tree node implementation details.
- Node represented as an object:
 - `node.state`: contains the state description.
 - `node.parent`: reference (pointer) to parent node.
 - `node.action`: the action taken by parent to get to this node's state.
 - `node.path_cost`: the full cost of the path from root to this node.
- Search tree is just a set of nodes created in memory. Nodes contain references (pointers) to parent nodes. This gives us a path from any node to the root node.
- Thus, a solution path can be constructed by a node that has a goal state.

Search Tree Node [2]

```
class Node:
    def __init__(self, state, parent=None, action_from_parent=None, path_cost=0):
        self.state = state
        self.parent = parent
        self.action_from_parent = action_from_parent
        self.path_cost = path_cost

        self.depth = 0
        if parent is not None:
            self.depth = parent.depth + 1
```

Best-first search template is **graph-like**

function BEST-FIRST-SEARCH(*problem*, *f*) **returns** a solution node or *failure*

node $\leftarrow$ NODE(STATE=*problem*.INITIAL)

frontier $\leftarrow$ a priority queue ordered by *f*, with *node* as an element

reached $\leftarrow$ a lookup table, with one entry with key *problem*.INITIAL and value *node*

while not IS-EMPTY(*frontier*) **do**

node $\leftarrow$ POP(*frontier*)

if *problem*.IS-GOAL(*node*.STATE) **then return** *node*

for each *child* **in** EXPAND(*problem*, *node*) **do**

s $\leftarrow$ *child*.STATE

if *s* is not in *reached* **or** *child*.PATH-COST < *reached*[*s*].PATH-COST **then**

reached[*s*] $\leftarrow$ *child*

add *child* to *frontier*

return *failure*

function EXPAND(*problem*, *node*) **yields** nodes

s $\leftarrow$ *node*.STATE

for each *action* **in** *problem*.ACTIONS(*s*) **do**

s' $\leftarrow$ *problem*.RESULT(*s*, *action*)

cost $\leftarrow$ *node*.PATH-COST + *problem*.ACTION-COST(*s*, *action*, *s'*)

yield NODE(STATE=*s'*, PARENT=*node*, ACTION=*action*, PATH-COST=*cost*)

- So, we go in loops, on each iteration, we select the node from the frontier with min $f(n)$.
- Frontier can be implemented as a priority-queue with priority defined by the evaluation function $f(n)$.
- We check for repeated states. So, children are added to frontier if they have not been reached, or if the path to reach this state is better than what we currently know.
- Reached is a set of states that have already been visited. Can be implemented as a lookup table *reached*[*s*], the keys are states and values are the *best* node for that state. (python dictionary)
- If the frontier is empty, this means we've exhausted all possible paths, and none give us a path to the goal. This means there is no solution (failure).
- **Is this a graph or tree-like search?**
- How to convert it to tree-like search?

Best-first search template – graph-like vs **tree-like**

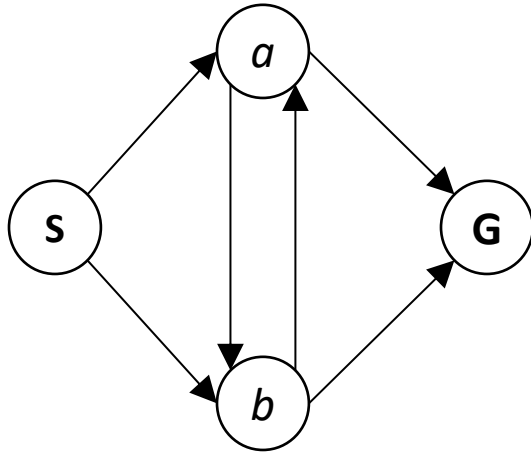
function BEST-FIRST-SEARCH(*problem*, *f*) **returns** a solution node or *failure*
 node $\leftarrow$ NODE(STATE=*problem*.INITIAL)
 frontier $\leftarrow$ a priority queue ordered by *f*, with *node* as an element
 ~~*reached* $\leftarrow$ a lookup table, with one entry with key *problem*.INITIAL and value *node*~~
 while not IS-EMPTY(*frontier*) **do**
 node $\leftarrow$ POP(*frontier*)
 if *problem*.IS-GOAL(*node*.STATE) **then return** *node*
 for each *child* **in** EXPAND(*problem*, *node*) **do**
 s $\leftarrow$ *child*.STATE
 ~~**if** *s* is not in *reached* **or** *child*.PATH-COST $<$ *reached*[*s*].PATH-COST **then**~~
 ~~*reached*[*s*] $\leftarrow$ *child*~~
 add *child* to *frontier*
 return *failure*

function EXPAND(*problem*, *node*) **yields** nodes
 s $\leftarrow$ *node*.STATE
 for each *action* **in** *problem*.ACTIONS(*s*) **do**
 s' $\leftarrow$ *problem*.RESULT(*s*, *action*)
 cost $\leftarrow$ *node*.PATH-COST + *problem*.ACTION-COST(*s*, *action*, *s'*)
 yield NODE(STATE=*s'*, PARENT=*node*, ACTION=*action*, PATH-COST=*cost*)

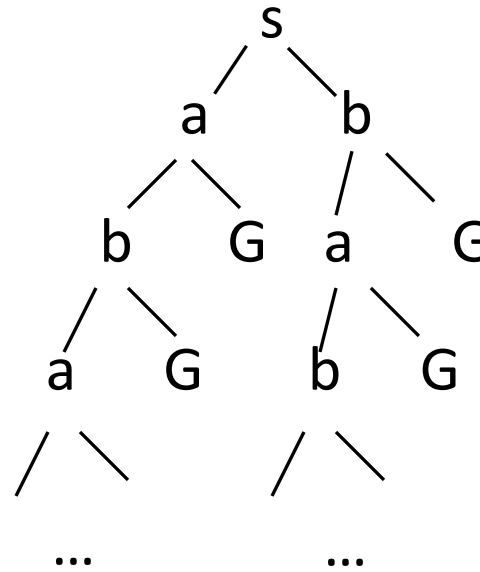
- How to convert it to tree-like search?
- If we remove all references to *reached*.
- In tree-like search we do not check for repeated states, so cycles and infinite paths can occur.
- Important to note this difference between graph-like vs tree-like search because we refer to it when we talk about algorithm properties.
- Note that with graph-like search, maintaining *reached* incurs storage penalty.

Graph-like vs tree-like search

Consider this 4-state graph:



How big is its search tree (from S)?



- We want to keep track of states that have been **reached** to prevent such redundant paths/cycles. A search algorithm that does this is said to perform a **graph-like search**.
- If the search algorithm **allows** redundant/cycle paths, it is said to perform **tree-like search**.
- When we refer to best-first search template by default we mean graph-like unless said otherwise.

Search Algorithms [1]

- How to change the behavior of a search algorithm using the best-first search template? We just need to use different evaluation functions $f(n)$.
- We will discuss two categories of search algorithms.
- **Uninformed** search algorithms: no information about states beyond what is given in the problem definition.
 - Breadth First Search (BFS)
 - Uniform Cost Search (UCS)
 - Depth First Search (DFS)
 - Depth Limited Search (DLS)
 - Iterative Deepening Search (IDS)
 - Bidirectional Search (BIDS)
- **Informed (heuristic)** search: knows whether one non-goal state is more promising than another.
 - Greedy search, A* search, hill-climbing search, meta-heuristics

Search Algorithms [2]

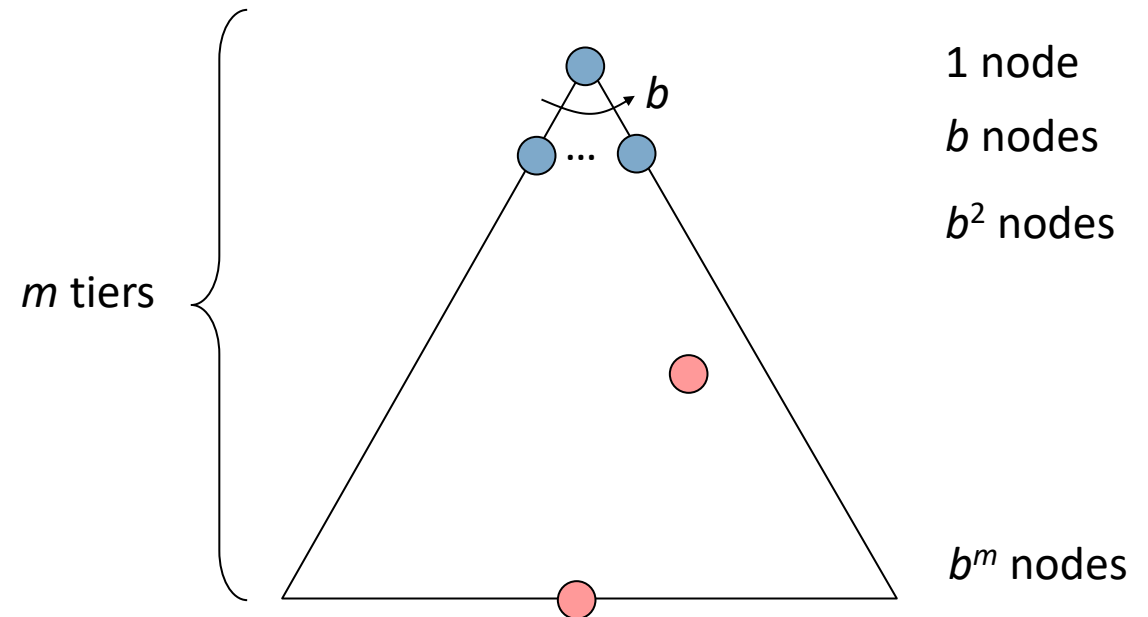
- How to evaluate/compare various search algorithms? We can use the following criteria:
 - **Completeness:** is it guaranteed to find a solution if it exists, and does it report a failure if a solution does not exist?
 - **Optimality:** does it find the minimal cost solution?
 - **Time complexity:** big-O time complexity in terms of:
 - Maximum number of nodes generated.
 - Problem's state space graph $|V| + |E|$ (# states and # actions).
 - **Space complexity:** big-O space complexity in terms of:
 - Maximum number of nodes in memory.
 - Problem's state space graph $|V| + |E|$ (# states and # actions).

Search Algorithms – time and space complexity terms

Time complexity = number of nodes generated during the search

Space complexity = max. number of nodes stored in memory at any point in time

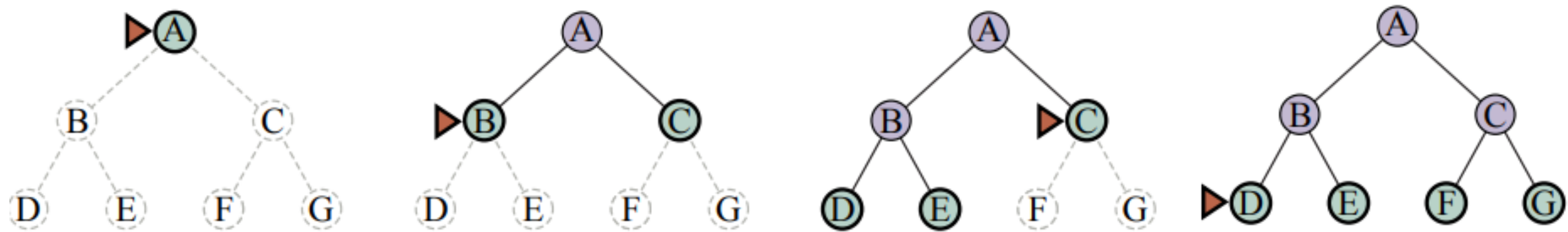
- b is the branching factor. This is the maximum number of actions among all states.
- d is depth of *shallowest* goal node. You may not know this *exactly* in practice.
- m is the maximum depth of *any* path in the state space (maybe unknown exactly).
 - Tree-like search: can be infinite if state-space graph has cycles.
 - Graph-like search: since redundancy is prevented, m can be smaller than in tree-like search.



$$\text{Total nodes} = 1 + b + b^2 + \dots + b^m = O(b^m)$$

Uninformed Search Algorithms

Breadth-first search [1]

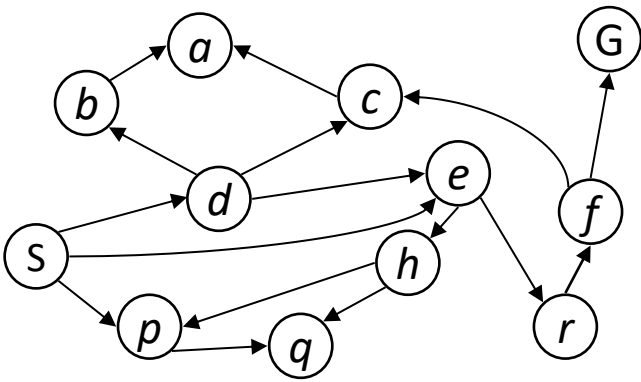


- Expand the **shallowest** unexpanded node, starting with the root, its successors, then their successors, and so on.
- If we wanted to use priority-queue in best-first search template, what is the evaluation function $f(n)$? It is the depth of the node in the search tree.
- The **frontier** can be implemented as a FIFO queue (more efficient than priority queue) . Generated successors are added at the end of the frontier queue.

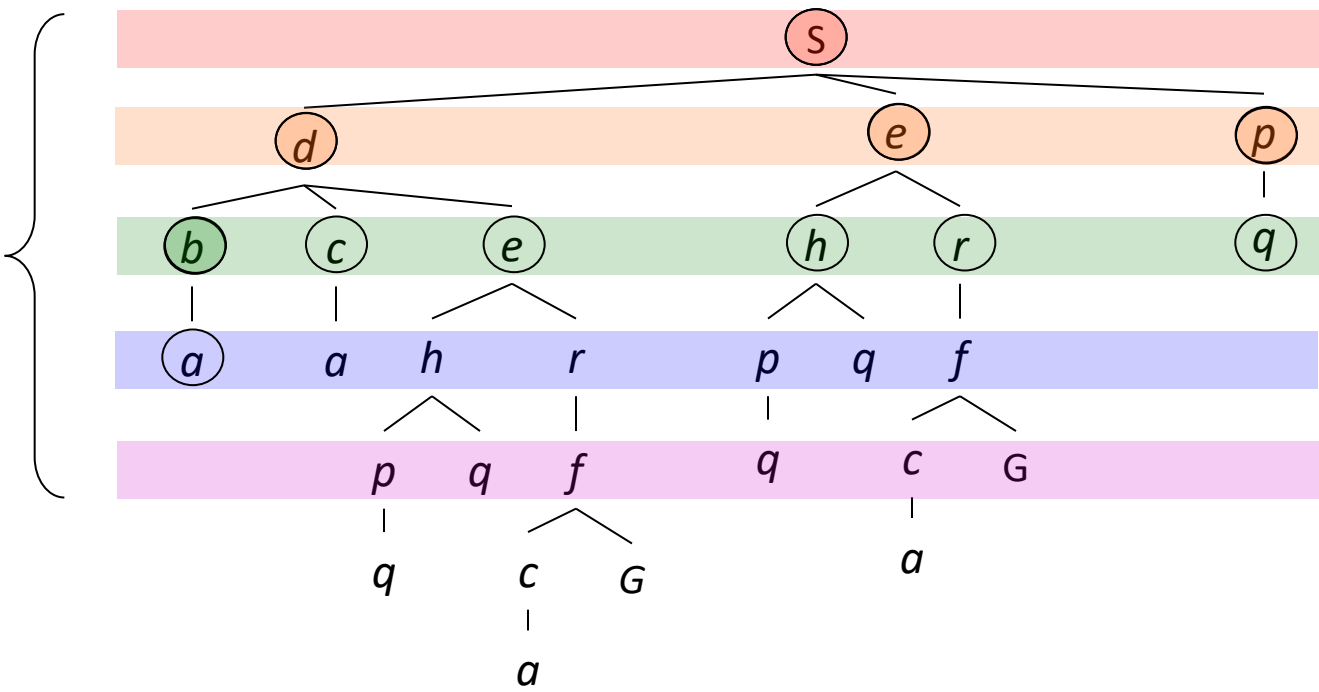
Breadth-first search [2]

Strategy: expand a shallowest node first

Implementation:
Frontier is a FIFO queue

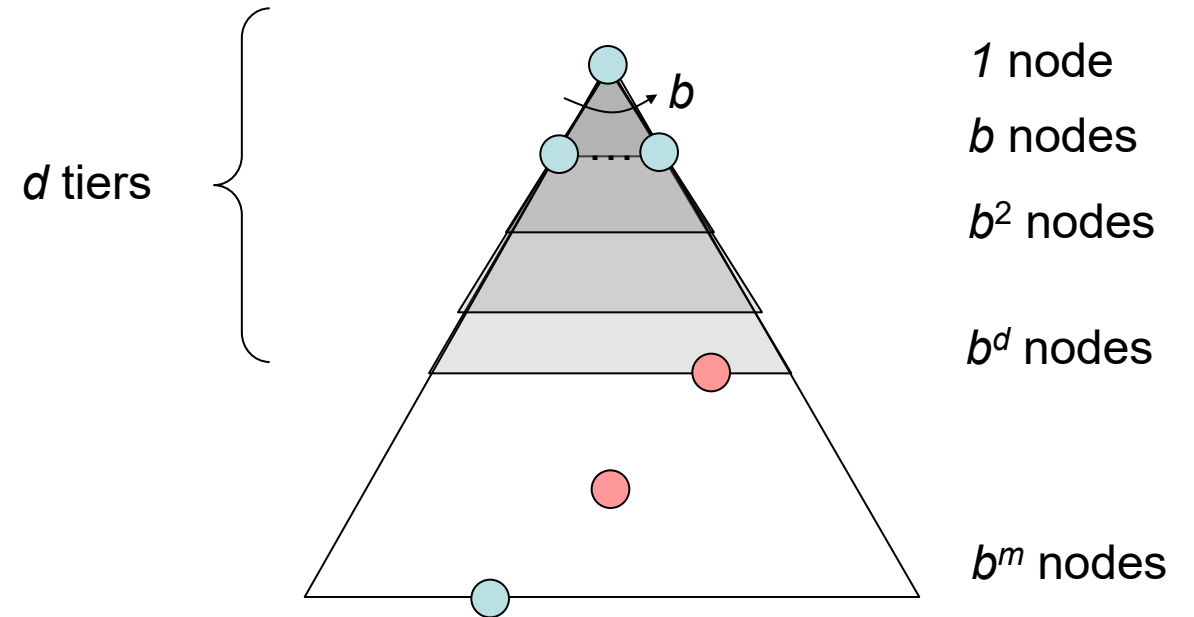


Search Tree
Tiers



Breadth-first search [3]

- What nodes does BFS expand?
 - Processes all nodes above shallowest goal
 - Let depth of shallowest solution be d
- Time complexity?
 - $O(b^d)$
- Space complexity?
 - $O(b^d)$
- Is it complete?
 - yes
- Is it optimal?
 - Only if action costs are equal.



Time complexity = number of nodes generated during the search

Space complexity = max. number of nodes stored in memory

- b is the branching factor. This is the maximum number of successors of any node.
- d is depth of *shallowest* goal node.
- m is the maximum depth of *any* path in the state space.

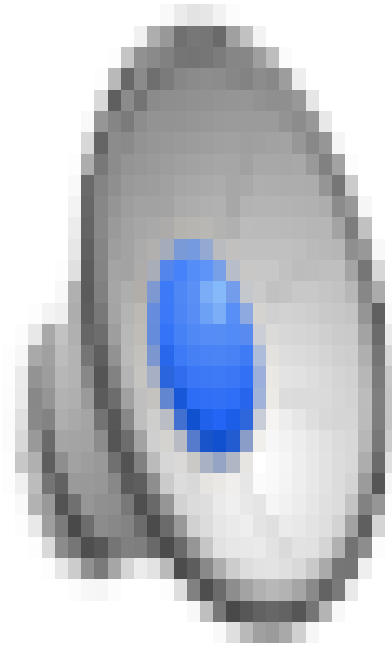
Breadth-first search [4] – Time and Space Example

- Assume $b = 10$; a computer that can generate 1M nodes/second; 1000 bytes/node

d	Nodes	Time $O(b^d)$	Memory $O(b^d)$
2	111	0.11 msec	107 k
4	11111	11 msec	10.6M
6	$O(10^6)$	1.1 sec	1 G
8	$O(10^8)$	2 min	103 G
10	$O(10^{10})$	3 hrs	10 T
12	$O(10^{12})$	13 days	1 Peta
16	$O(10^{16})$	350 yrs	10 Exa

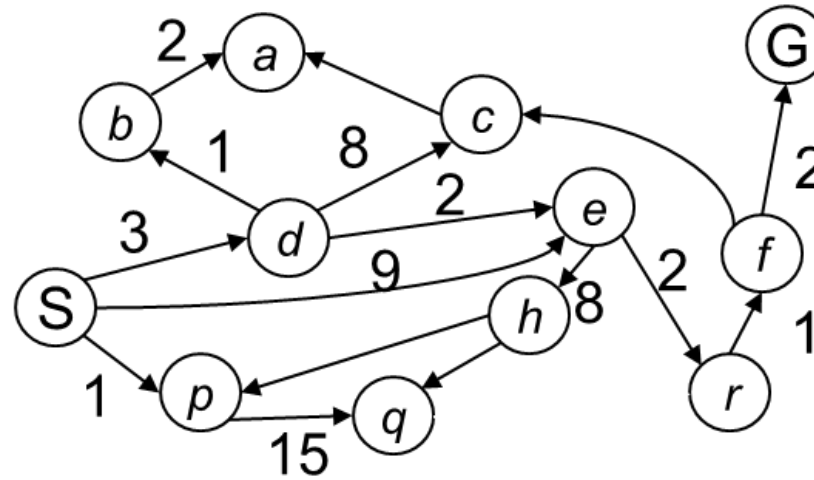
- Space is the bigger problem (more than time) for BFS, but time still a major factor

BFS search behavior in grid environment



Video from [link](#)

Uniform-cost search [1]



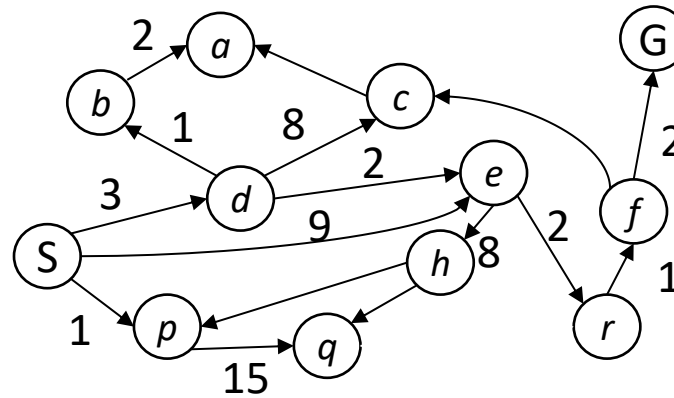
- BFS finds the shortest path in terms of number of actions (not least cost path). On the other hand, UCS does find the least cost path.
- Uses priority queue, expand node n with lowest path cost $g(n)$. Thus, in the best-first search template, set $f(n) = g(n)$.
- Keep track of already visited states to see if a lower cost path is found to that state.
- If we assume that action costs are non-negative, then UCS is optimal for any action cost function.
- Becomes BFS if all action costs are equals.

Uniform-cost search [2]

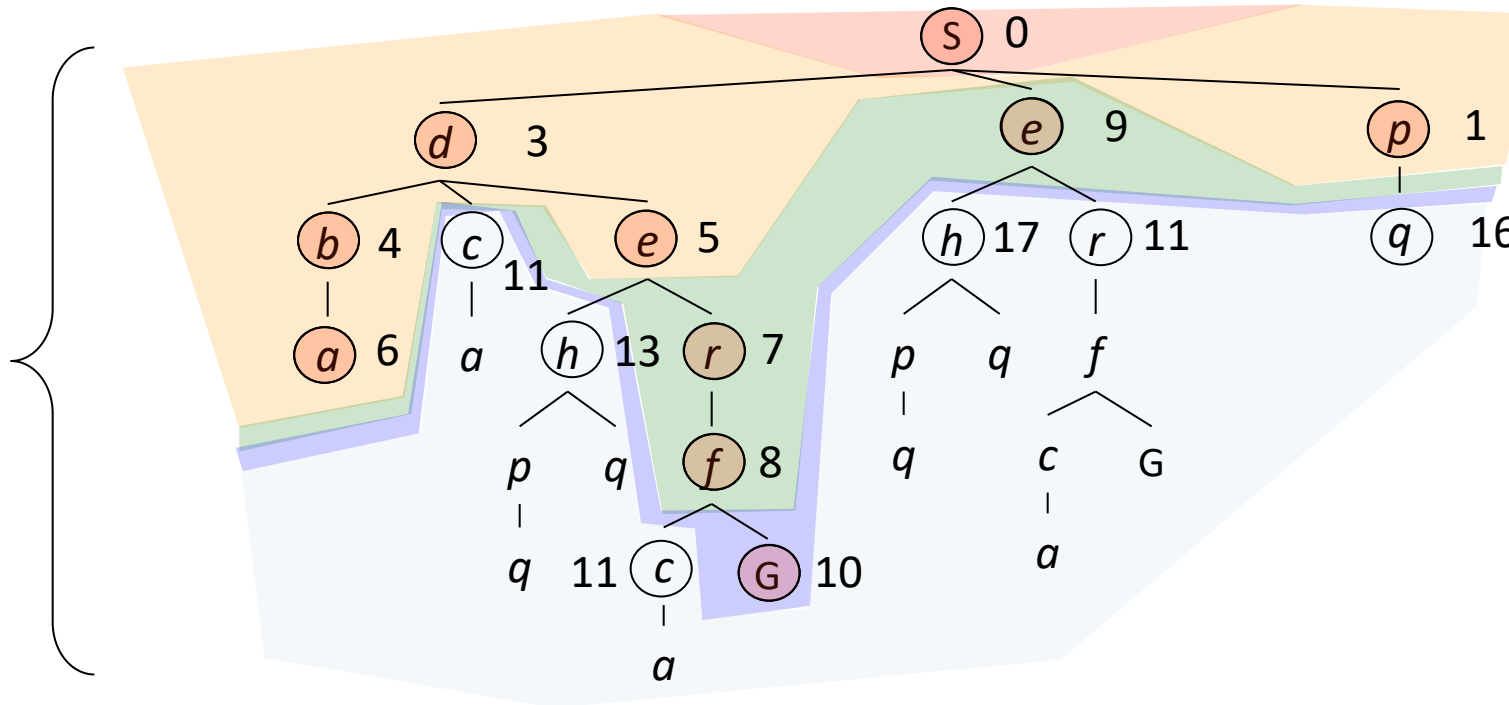
$g(n)$ = path-cost from root to n

Strategy: expand lowest $g(n)$

Frontier is a priority queue sorted by $g(n)$

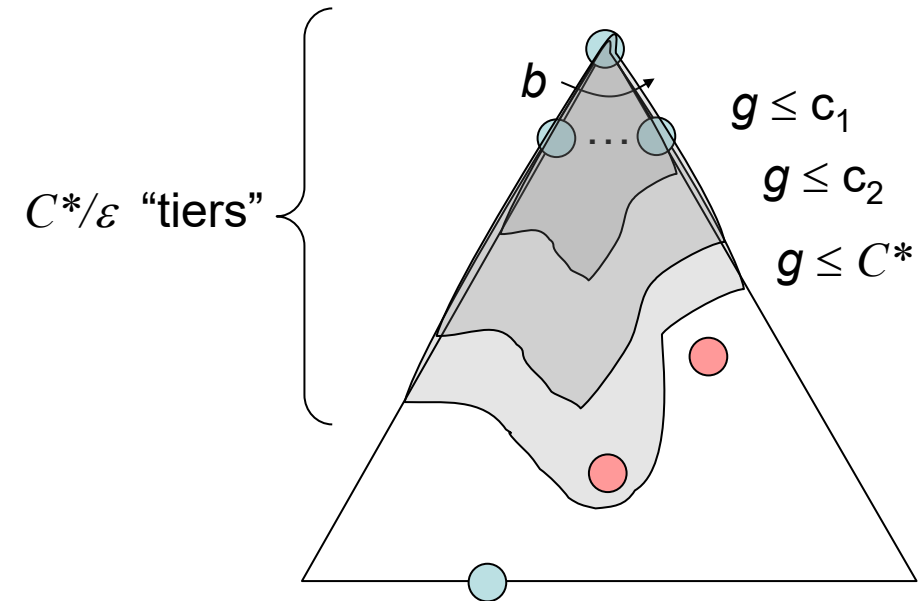


Search Tree
With cost
contours



Uniform-cost search [3]

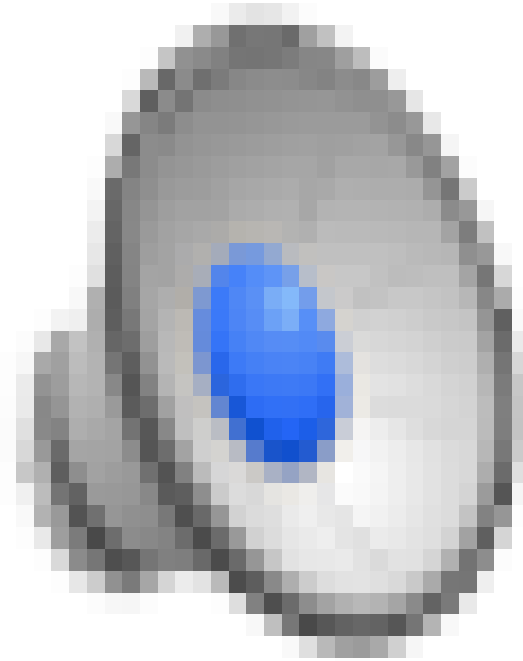
- What nodes does UCS expand?
 - Processes all nodes with cost less than cheapest solution, C^*
 - If actions cost at least ε then the “effective depth” is roughly $1 + \lfloor C^* / \varepsilon \rfloor$
- Time complexity?
 - $O(b^{1 + \text{floor}(C^* / \varepsilon)})$ exponential in effective depth
- Space Complexity?
 - $O(b^{1 + \text{floor}(C^* / \varepsilon)})$
- Is it complete?
 - Assuming C^* is finite and $\varepsilon > 0$, yes.
- Is it optimal?
 - Yes.



Time complexity = number of nodes generated during the search

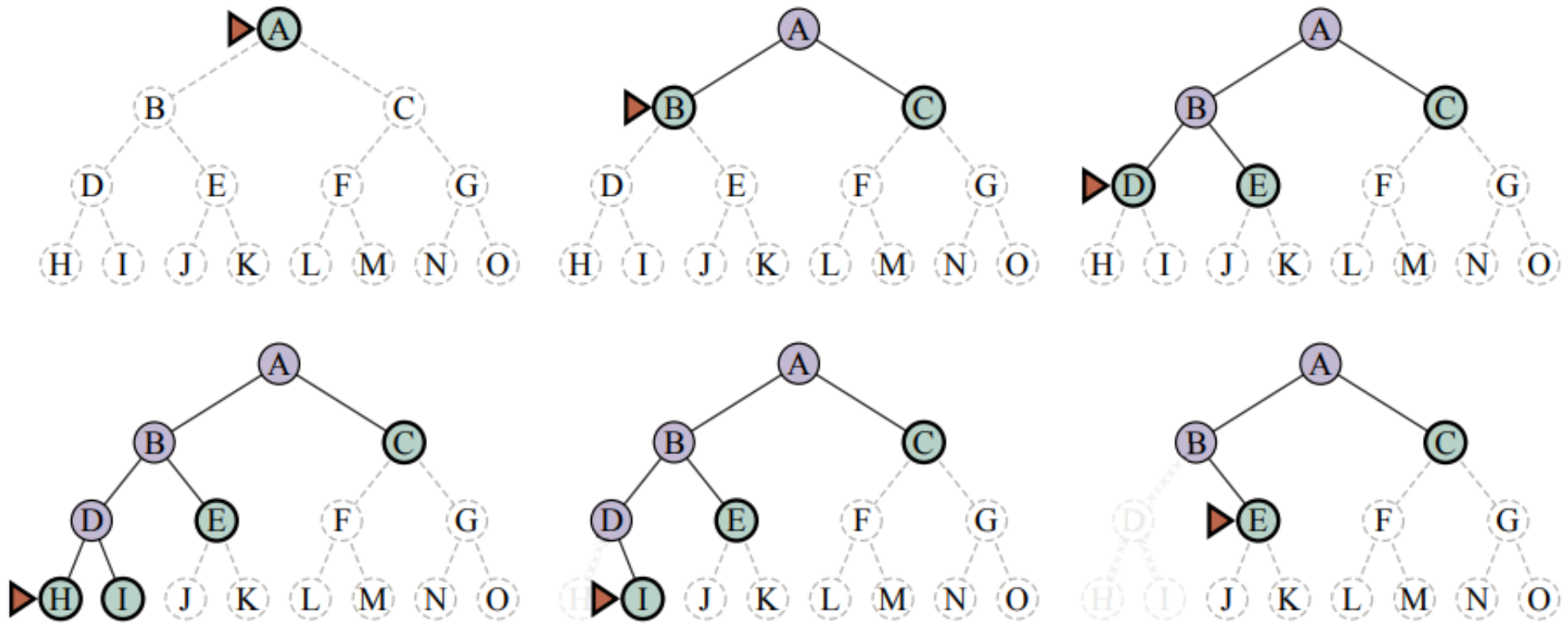
Space complexity = max. number of nodes stored in memory

UCS search behavior in grid environment



Video from [link](#)

Depth-first search [1]

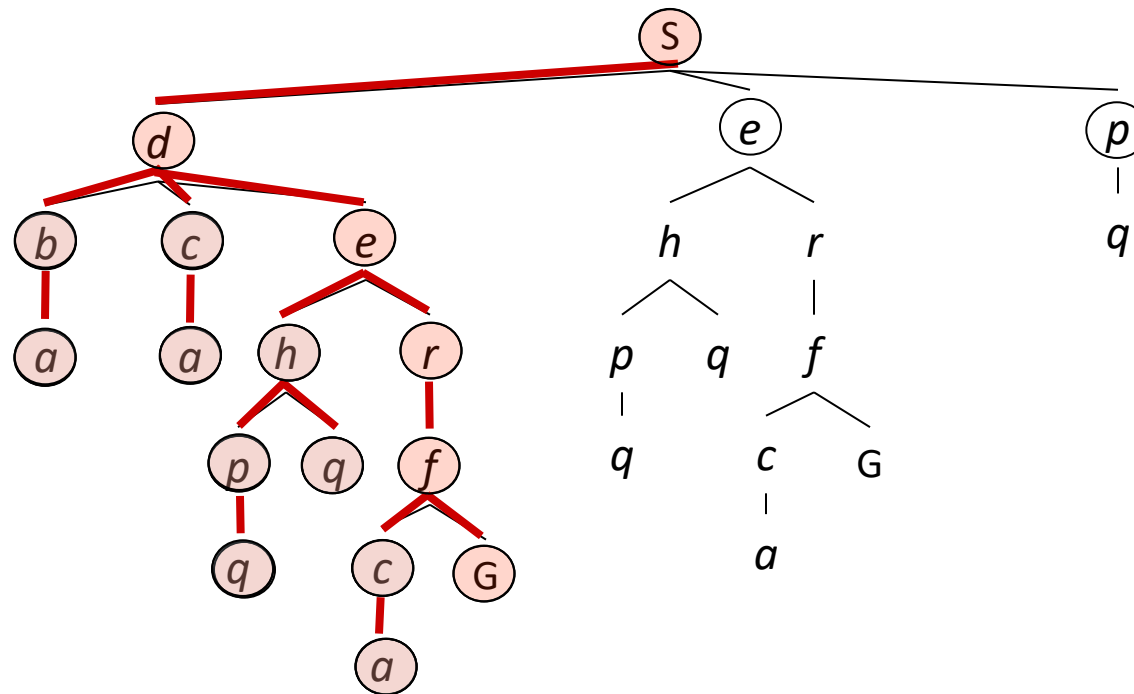
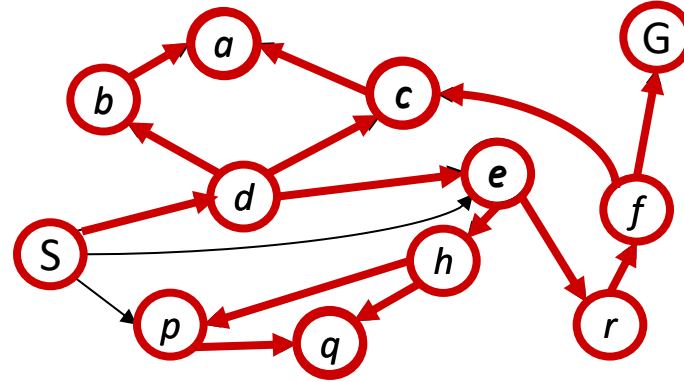


- Expand the **deepest** unexpanded node.
- If we wanted to use priority-queue in best-first search template, what is the evaluation function $f(n)$? It is the negative of node depth.
- The frontier can be implemented as a LIFO queue. Generated successors are at front of queue. (more efficient than priority queue)

Depth-first search [2]

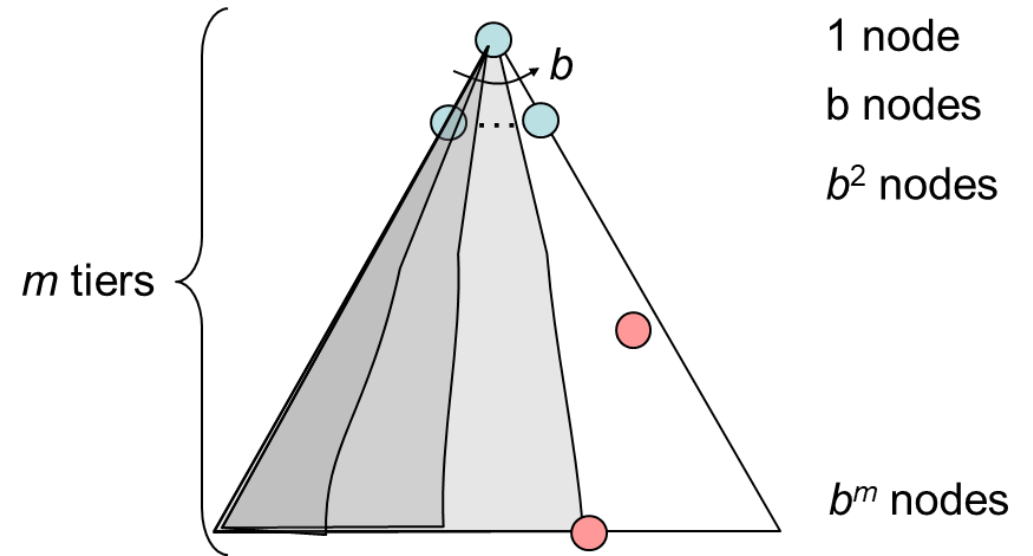
Strategy: expand a deepest node first

Implementation: Frontier is a LIFO queue



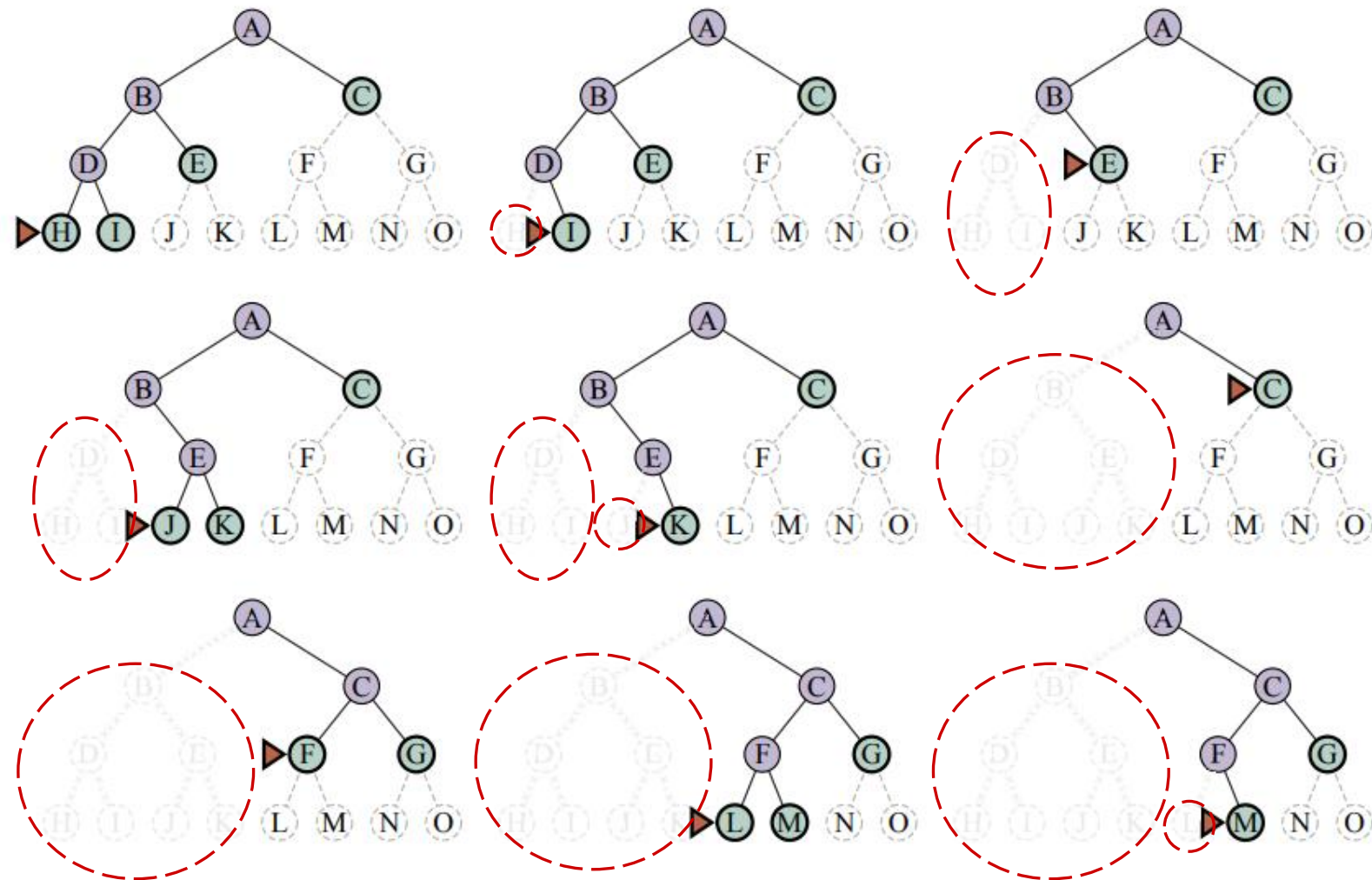
Depth-first search [3]

- What nodes does DFS expand?
 - Some left prefix of the tree down to depth m .
 - Could process the whole tree.
- Time complexity?
 - If m is finite, then in the worst case the goal is at the right most bottom node. Thus, it is $O(b^m)$
- Space Complexity?
 - Tree-like search: $O(bm)$ linear in b and m .
 - Graph-like search: $O(|V|)$ number of states to maintain in *reached*. This can be much larger than $O(bm)$.
 - This is why typically DFS is implemented and used in practice as a tree-like search; i.e. it does not maintain *reached*. This is done so that it can reduce space complexity.
- Is it complete?
 - No, if state space is infinite (both tree-like and graph-like DFS).
 - No, if cycles are not checked (tree-like DFS).
 - Yes, if cycles are prevented and state space is finite (graph-like DFS).
- Is it optimal?
 - No, it finds the “leftmost” solution, regardless of depth or cost.



Depth-first search – tree-like space complexity

- Assume finite search trees and we use tree-like DFS.
- Worst case** memory is illustrated by the first image. At each depth from 1 to m , we will have b children. So, roughly $b + b + \dots + b$ (m -times).
- $O(bm)$ linear in b and m .
- Unlike breadth-first search, we do not need to store parts of the search tree that were already fully explored.
- These parts of the tree did not contain a solution, so we can safely remove it from memory.
- Main takeaway, **tree-like DFS saves a lot of space**.
- Might not be the case for graph-like DFS



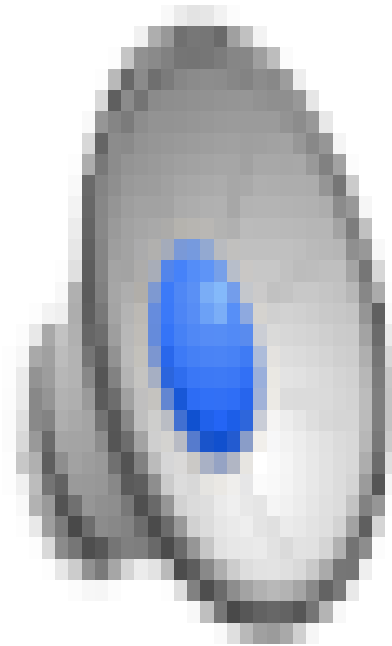
Tree-like Depth-first search – Analysis

- Assuming $b = 10$; $m = 12$; 10000 nodes/sec; 1kbyte/node. d is depth of solution

d	Nodes	Time (worst-case)	Memory (worst-case)
2	10^{12}	3 years	120kb
4	10^{12}	3 years	120kb
8	10^{12}	3 years	120kb
12	10^{12}	3 years	120kb

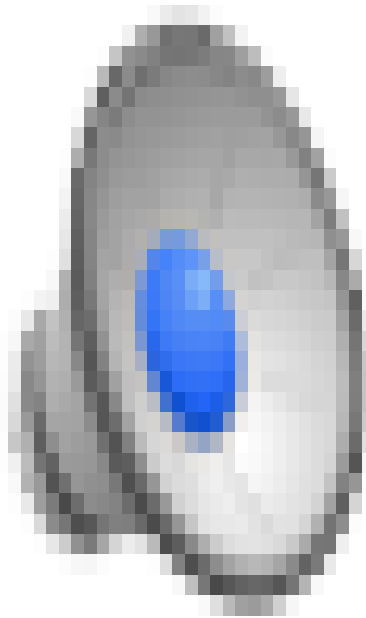
- Main takeaway, tree-like DFS saves a lot of space. Used as basic workhorse traversal of many areas of AI: constraint satisfaction, propositional satisfiability, and logic programming.
- But can be terrible if m is much larger than depth of solution.
- Also not guaranteed to find the optimal solution.
- m can be infinite. So may never terminate along a path.

DFS search behavior in grid environment



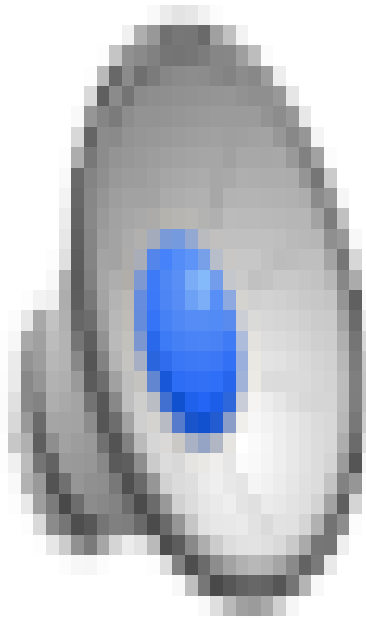
Video from [link](#)

Which algorithm behavior? BFS, UCS, or DFS
Note: dark blue grids have higher cost than light blue



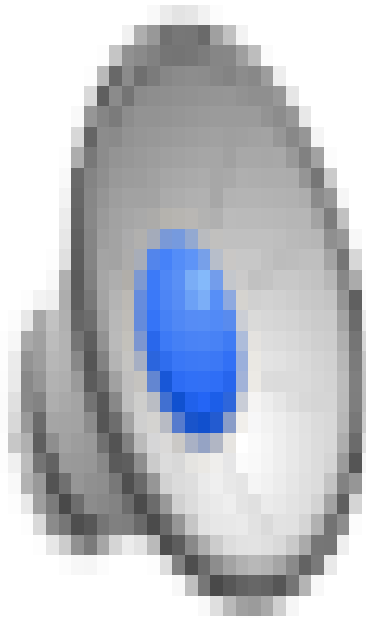
Video from [link](#)

Which algorithm behavior? BFS, UCS, or DFS
Note: dark blue grids have higher cost than light blue



Video from [link](#)

Which algorithm behavior? BFS, UCS, or DFS
Note: dark blue grids have higher cost than light blue



Video from [link](#)

Depth-limited DFS

- Applies DFS, but search tree is not explored below some depth limit L .
- Reduces time complexity from $O(b^m)$ to $O(b^L)$.
- With tree-like DFS, space complexity is $O(bL)$. With graph-like DFS, space complexity can still be $O(|V|)$.
- Solves problem of infinitely deep paths, but will return failure if solution is below depth limit.
- How to select depth limit L ?
 - Based on problem knowledge, e.g. diameter of state space. E.g ., max number of steps between any 2 cities.
 - But, typically not known ahead of time in practice.
- Is complete?
 - If $d \leq L$, then yes.
 - If $d > L$, then no. In general, we do not know depth of d .
- Is Optimal? No. Could find non-optimal solution before optimal solution.

Iterative-deepening DFS (IDS) [1]

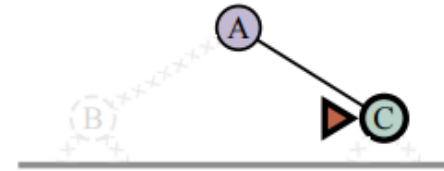
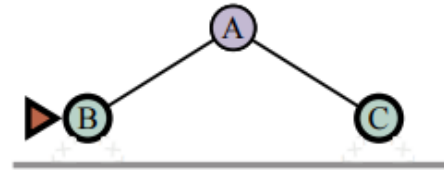
- IDS solves issue of selecting L in depth-limited search.
- Performs depth-limited DFS at various depths $L = 1, 2, 3, \dots$
- Combines benefits of BFS and DFS.
- Has DFS's space complexity:
 - Tree-like DFS $O(bd)$ when there is solution, or $O(bm)$ when no solution.
 - Graph-like DFS can still be $O(|V|)$.
- Like BFS:
 - Optimal when all actions have the same cost.
 - Complete if finite state spaces and cycles are prevented (graph-like). Otherwise, can run to $L = \infty$

Iterative-deepening DFS (IDS) [2]

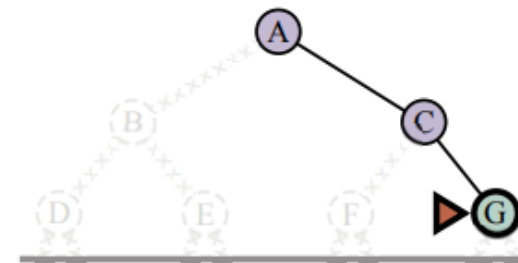
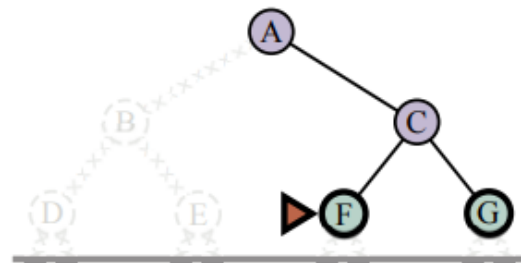
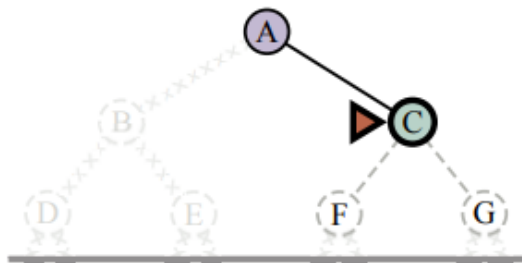
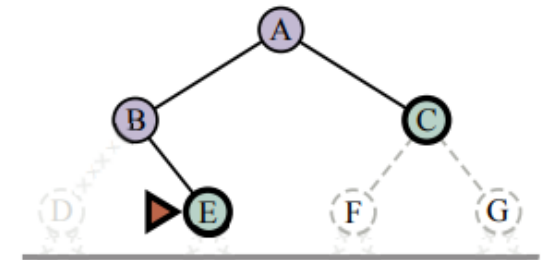
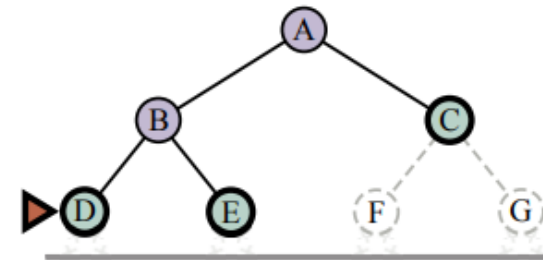
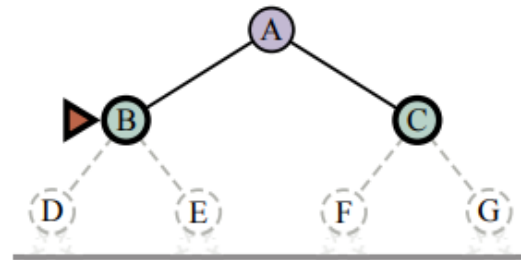
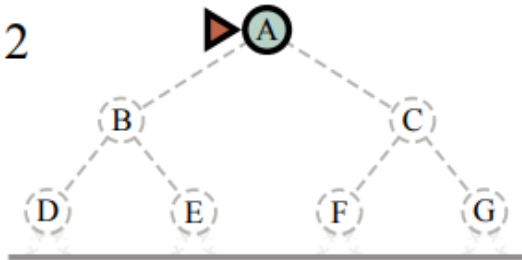
limit: 0



limit: 1

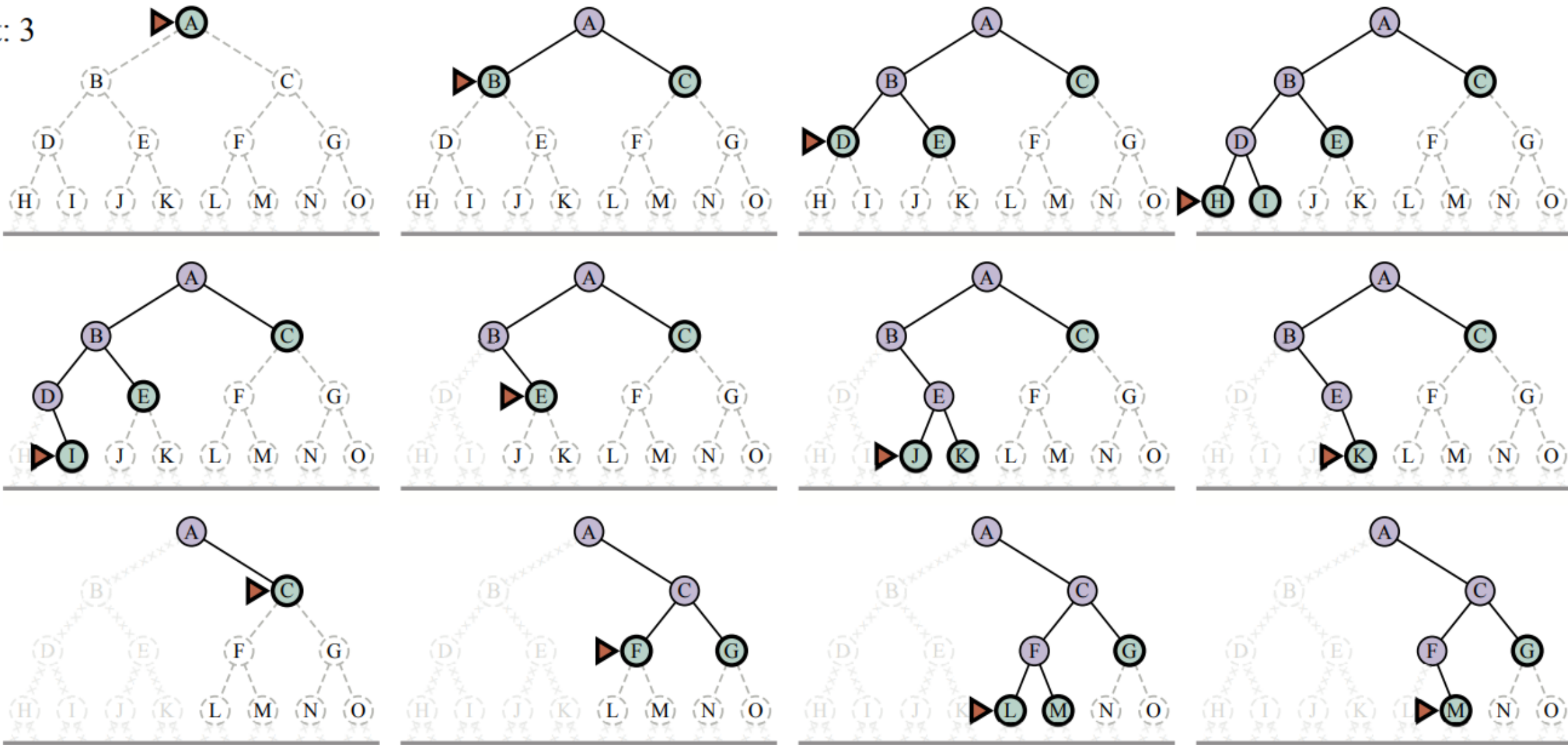


limit: 2



Iterative-deepening DFS (IDS) [3]

limit: 3



Iterative-deepening DFS (IDS) [4]

- Has DFS's space complexity:
 - Tree-like DFS $O(bd)$ when there is solution, or $O(bm)$ when no solution.
 - Graph-like DFS can still be $O(|V|)$ which can be larger than $O(bd)$ or $O(bm)$.
- Like BFS:
 - Optimal when all actions have the same cost.
 - Complete if finite state spaces and cycles are prevented (graph-like). Otherwise, can run to $L = \infty$
- Time complexity:
 - Like BFS, $O(b^d)$ when there is solution, or $O(b^m)$ when no solution.
 - Asymptotically same as BFS or DFS limited to depth d in the worst case.
 - But, in practice it is a little worse than BFS or DFS. Why?
 - Because nodes near the top of the search tree are generated multiple times for each depth limit.
- IDS with tree-like DFS is generally preferred method for large state spaces where solution depth d is unknown. Tree-like DFS is used to save memory.

Iterative-deepening DFS (IDS) vs depth-limited DFS (DLS)

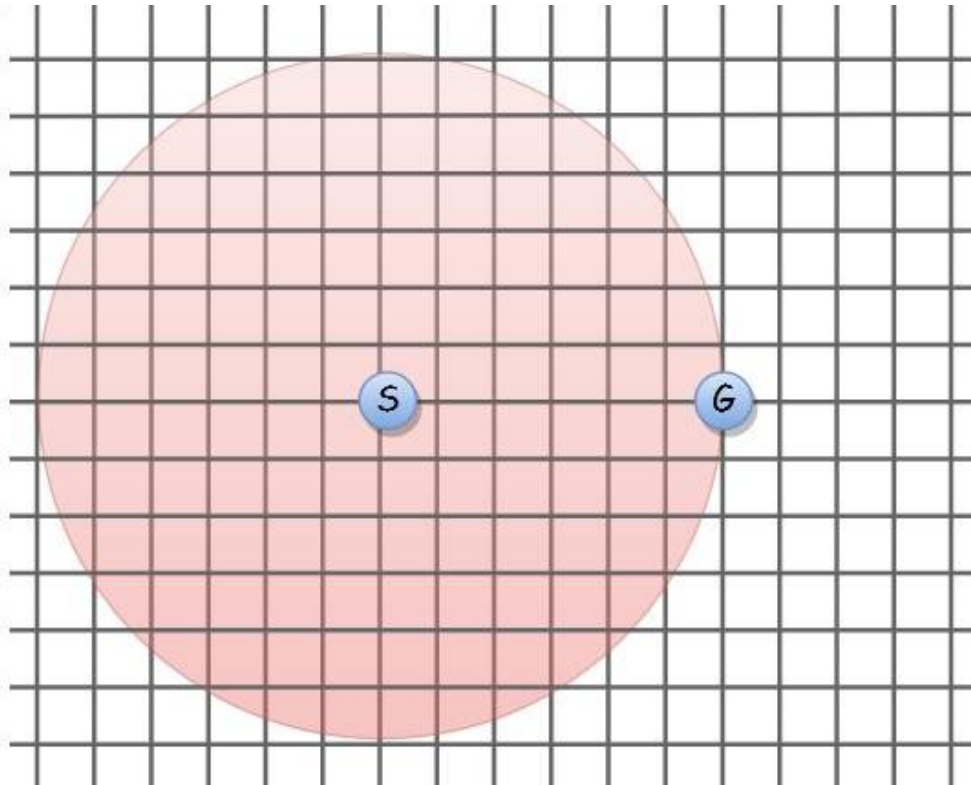
- Both IDS and DLS **asymptotic** time complexity is $O(b^d)$ nodes. But practically, IDS generates more nodes.
- Number of nodes generated with depth d and branching factor b .
 - $N(DLS) = b^1 + b^2 + \dots + b^{d-2} + b^{d-1} + b^d$
 - $N(IDS) = (d)b^1 + (d-1)b^2 + \dots + 3b^{d-2} + 2b^{d-1} + b^d$
- Example: For $b = 10$, $d = 5$
 - $N(DLS) = 10 + 100 + 1000 + 10000 + 100000 = 111110$
 - $N(IDS) = 50 + 400 + 3000 + 20000 + 100000 = 123450$
- $Overhead = (123450 - 111110) / 111110 = 11\%$

Bidirectional Search [1]

- Idea behind bidirectional search is to simultaneously perform two searches:
 - One searches forward from the initial state to the goal state.
 - Another searches backward from the goal state to the initial state.
 - Stop when both searches “meet”.
- Conceptually we are searching on two problems where one is the inverse of the other.
- Need to keep track of two frontiers. If the frontiers intersect/collide, then a solution has been found.
- Motivation is that at the bottom search tree of the two searches, we will have $b^{d/2}$. So, time and space complexity is $O(b^{d/2})$.
- Implemented as two best-first searches with a forward evaluation function and backward evaluation function. See textbook for pseudocode.

Breadth-first search vs Bidirectional Search [2]

Breadth-first search expansion



Bidirectional search expansion

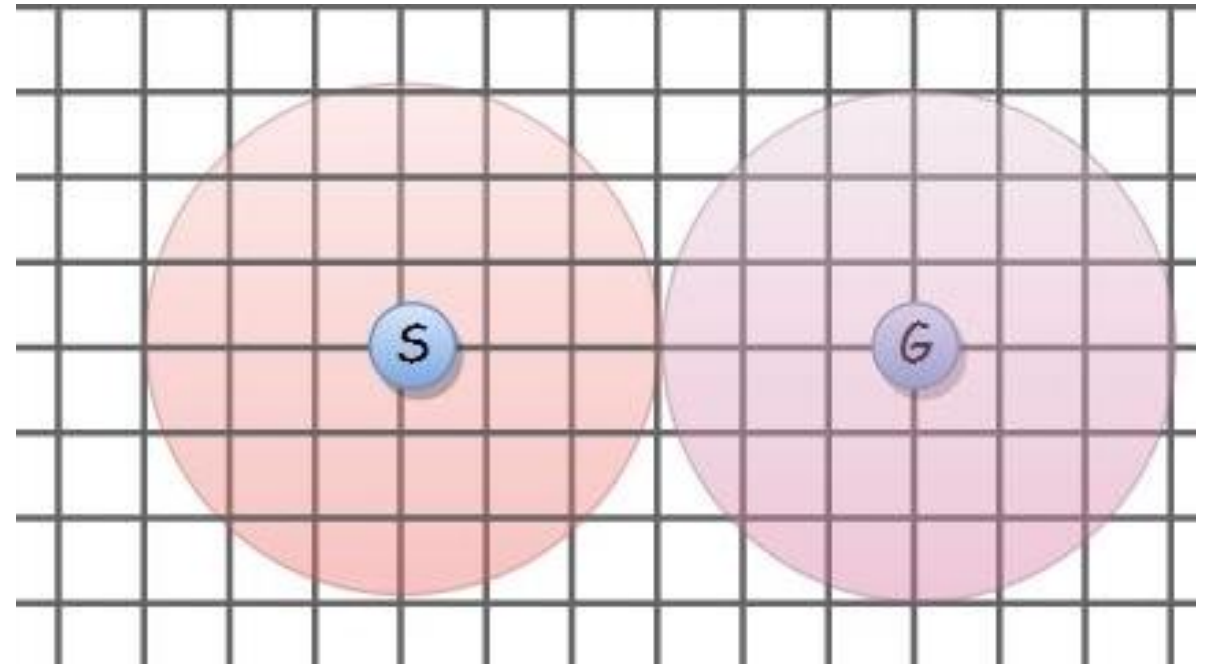


Image from: [link](#)

Uninformed Search Algorithms $f(n)$ Summary

- Recall that all the BFS, DFS, UCS, etc. can be used with the best-first search template with different evaluation function $f(n)$.

Algorithm	Evaluation function $f(n)$
BFS	depth of node
UCS	Actual path-cost from start to n
DFS	– (depth of node)
Depth-limited DFS	– (depth of node)
IDS-DFS	– (depth of node)

- How to implement this with the best-first search template code?

Uninformed Search Algorithms f(n) Summary

- How to implement this with the best-first search template code?
- Below is example of `ucs_search` that calls `best_first_search()` with correct arguments.

```
def ucs_search(problem, h):  
    ucs_f_func = (lambda node: node.path_cost)  
    return best_first_search(problem, f=ucs_f_func)
```

Uninformed Search Algorithms Summary

- Assuming we do not check for repeated states (tree-like search). The following table compares the uninformed searches.

Criterion	Breadth-First	Uniform-Cost	Depth-First	Depth-Limited	Iterative Deepening	Bidirectional (if applicable)
Complete?	Yes ¹	Yes ^{1,2}	No	No	Yes ¹	Yes ^{1,4}
Optimal cost?	Yes ³	Yes	No	No	Yes ³	Yes ^{3,4}
Time	$O(b^d)$	$O(b^{1+\lceil C^*/\epsilon \rceil})$	$O(b^m)$	$O(b^\ell)$	$O(b^d)$	$O(b^{d/2})$
Space	$O(b^d)$	$O(b^{1+\lceil C^*/\epsilon \rceil})$	$O(bm)$	$O(b\ell)$	$O(bd)$	$O(b^{d/2})$

Figure 3.15 Evaluation of search algorithms. b is the branching factor; m is the maximum depth of the search tree; d is the depth of the shallowest solution, or is m when there is no solution; ℓ is the depth limit. Superscript caveats are as follows: ¹ complete if b is finite, and the state space either has a solution or is finite. ² complete if all action costs are $\geq \epsilon > 0$; ³ cost-optimal if action costs are all identical; ⁴ if both directions are breadth-first or uniform-cost.

- If we do graph-like search, then main difference is DFS will be complete for finite state spaces and space complexity will be $O(|V|)$. Remaining time and space complexities will be same as written, but m (*max path depth*) will generally be smaller in a graph-like tree, so less search work.
- In the book, they use a looser bound for graph-like search. “In graph search, space and time complexities are bounded by the size of the state space (i.e. number of vertices and edges $O(|V| + |E|)$)”.

Uninformed Search Algorithms Summary

- Which algorithm to use in practice is problem specific.
- Some options to try:
 - Study your problem and identify theoretical bounds on b , d , m , ϵ , and C^* . This will be helpful in determining bounds on time and space costs. This can be difficult to do.
 - If compute resources are available and you can generate randomized instances of your problem, then you can instantiate small to large instances, run various search algorithms, gather stats on time/space usage and success/failure rates. This will give you statistics on which algorithm does well on your problem. With a lot of compute resources, you can run these in parallel.

Uninformed Search Practice

for each *child* **in** EXPAND(*problem*, *node*) **do**

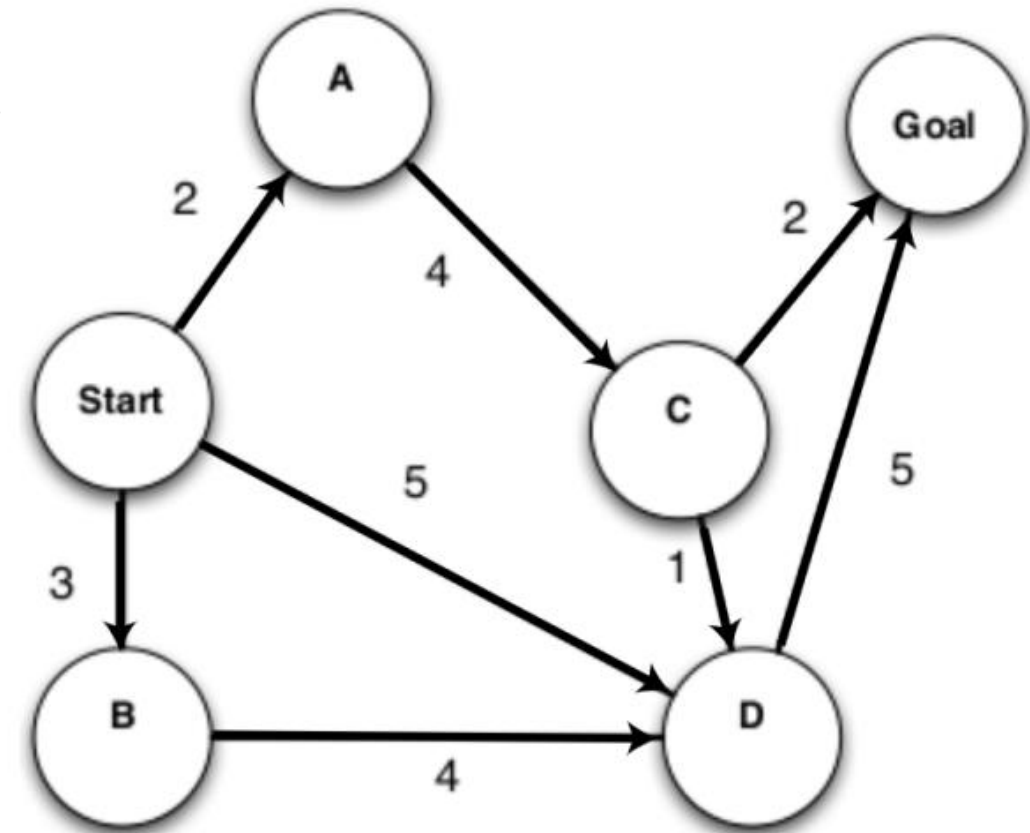
$s \leftarrow \text{child.STATE}$

if *s* is not in *reached* **or** *child.PATH-COST* < *reached*[*s*].PATH-COST **then**

$\text{reached}[s] \leftarrow \text{child}$

add *child* to *frontier*

- Apply BFS, DFS, UCS, and IDS. Use graph-like best-first search template. Assume ties to be resolved in alphabetical order.
- a) list the order in which states are popped from frontier
- b) solution path returned by each strategy and its cost to the goal.
- c) Total number of nodes popped from the frontier.
- d) Total number of nodes generated.



function BEST-FIRST-SEARCH(*problem*, *f*) **returns** a solution node or *failure*

node $\leftarrow$ NODE(STATE=*problem*.INITIAL)

frontier $\leftarrow$ a priority queue ordered by *f*, with *node* as an element

reached $\leftarrow$ a lookup table, with one entry with key *problem*.INITIAL and value *node*

while not IS-EMPTY(*frontier*) **do**

node $\leftarrow$ POP(*frontier*)

if *problem*.IS-GOAL(*node*.STATE) **then return** *node*

for each *child* **in** EXPAND(*problem*, *node*) **do**

s $\leftarrow$ *child*.STATE

if *s* is not in *reached* **or** *child*.PATH-COST < *reached*[*s*].PATH-COST **then**

reached[*s*] $\leftarrow$ *child*

add *child* to *frontier*

return *failure*

function EXPAND(*problem*, *node*) **yields** nodes

s $\leftarrow$ *node*.STATE

for each *action* **in** *problem*.ACTIONS(*s*) **do**

s' $\leftarrow$ *problem*.RESULT(*s*, *action*)

cost $\leftarrow$ *node*.PATH-COST + *problem*.ACTION-COST(*s*, *action*, *s'*)

yield NODE(STATE=*s'*, PARENT=*node*, ACTION=*action*, PATH-COST=*cost*)

- Recall this is the graph-like best-first search template.
- Recall that the tree-like best-first search variant removes all references to *reached* which does not prevent repetitions and cycles.
- For exam or quiz questions that ask you to list order of nodes being popped using **graph-like best-first search**, we mean this particular implementation.

function BEST-FIRST-SEARCH(*problem*, *f*) **returns** a solution node or *failure*

node $\leftarrow$ NODE(STATE=*problem*.INITIAL)

frontier $\leftarrow$ a priority queue ordered by *f*, with *node* as an element

~~*reached* $\leftarrow$ a lookup table, with one entry with key *problem*.INITIAL and value *node*~~

while not IS-EMPTY(*frontier*) **do**

node $\leftarrow$ POP(*frontier*)

if *problem*.IS-GOAL(*node*.STATE) **then return** *node*

for each *child* **in** EXPAND(*problem*, *node*) **do**

s $\leftarrow$ *child*.STATE

~~**if** *s* is not in *reached* **or** *child*.PATH-COST $<$ *reached*[*s*].PATH-COST **then**~~

~~*reached*[*s*] $\leftarrow$ *child*~~

add *child* to *frontier*

return *failure*

function EXPAND(*problem*, *node*) **yields** nodes

s $\leftarrow$ *node*.STATE

for each *action* **in** *problem*.ACTIONS(*s*) **do**

s' $\leftarrow$ *problem*.RESULT(*s*, *action*)

cost $\leftarrow$ *node*.PATH-COST + *problem*.ACTION-COST(*s*, *action*, *s'*)

yield NODE(STATE=*s'*, PARENT=*node*, ACTION=*action*, PATH-COST=*cost*)

- For exam or quiz questions that ask you to list order of nodes being popped using **tree-like best-first search**, we mean this implementation with the crossed out lines removed.
- This means that children are always added to frontier.

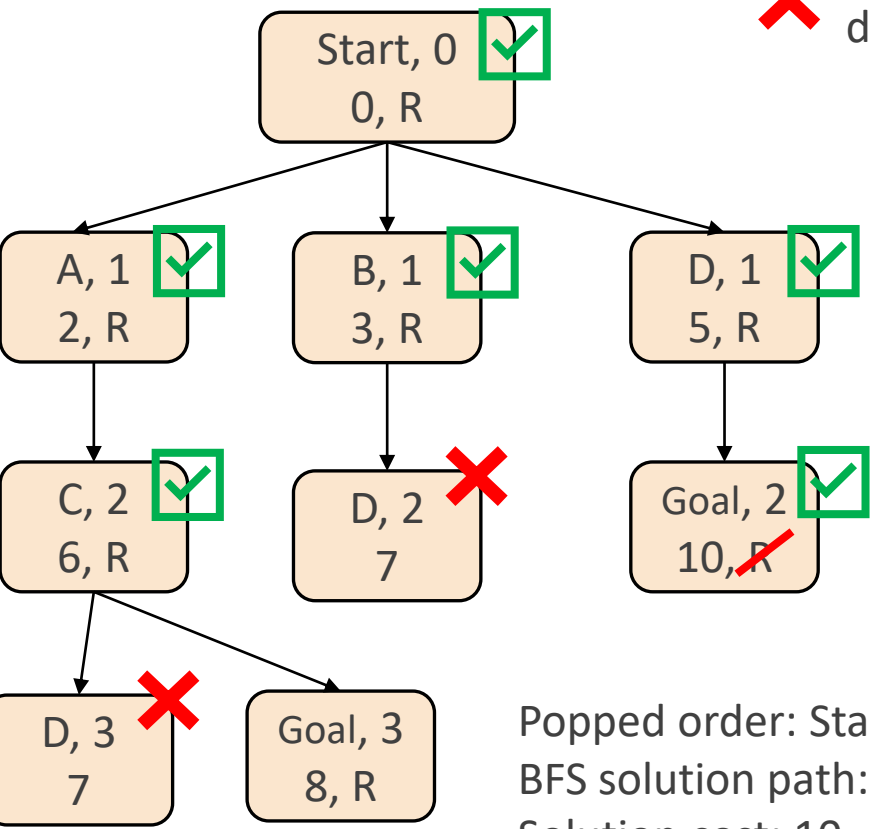
Uninformed Search Practice – BFS

```
for each child in EXPAND(problem, node) do
  s ← child.STATE
  if s is not in reached or child.PATH-COST < reached[s].PATH-COST then
    reached[s] ← child
    add child to frontier
```

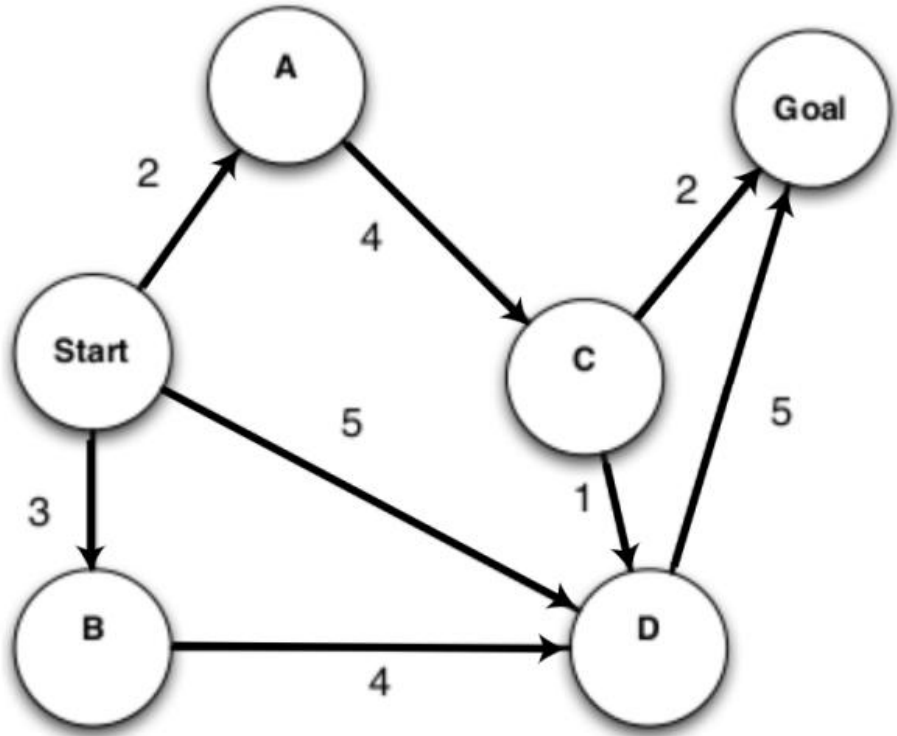
State, f-value
path-cost, Reached-pointer

✓ Denotes popped node

✗ Denotes node generated but discarded (not added to frontier)



Iter#	Popped
1	Start
2	A
3	B
4	D
5	C
6	Goal



Popped order: Start, A, B, D, C, Goal
BFS solution path: Start -> D -> Goal
Solution cost: 10
Total nodes popped: 6 Total nodes generated: 9

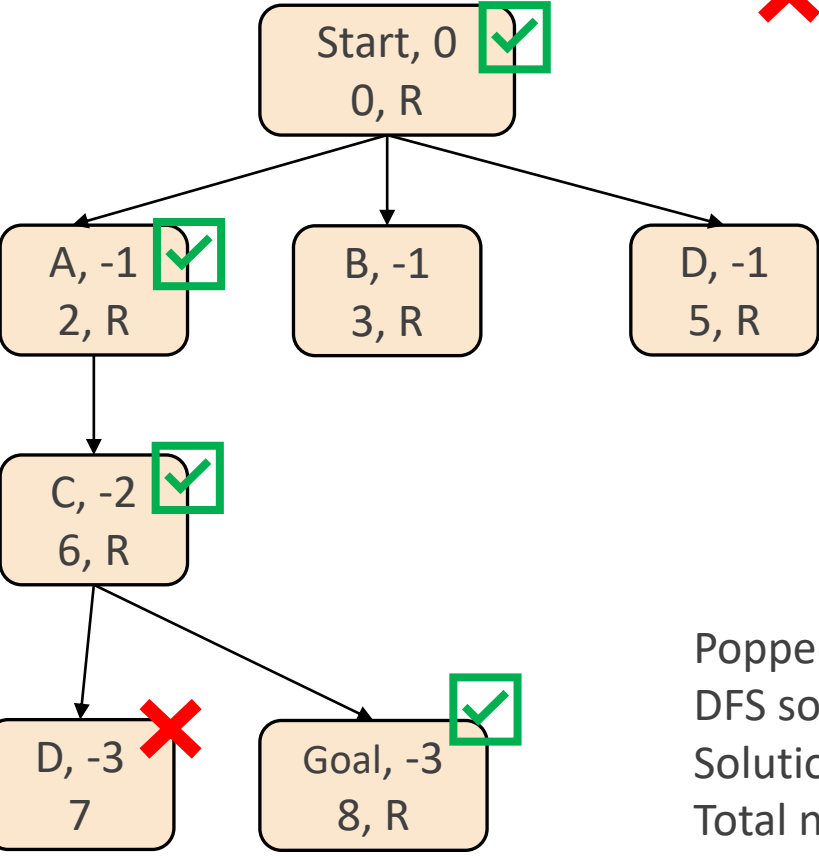
Note at iteration #3, the successor of B (which is D) is not added to frontier because we have already reached D via Start and it is shorter path. Similarly, with C at iteration #5, D is not added to frontier, but Goal is added since a shorter path was found and its reached-pointer is updated.

Uninformed Search Practice – DFS

```
for each child in EXPAND(problem, node) do
  s ← child.STATE
  if s is not in reached or child.PATH-COST < reached[s].PATH-COST then
    reached[s] ← child
    add child to frontier
```

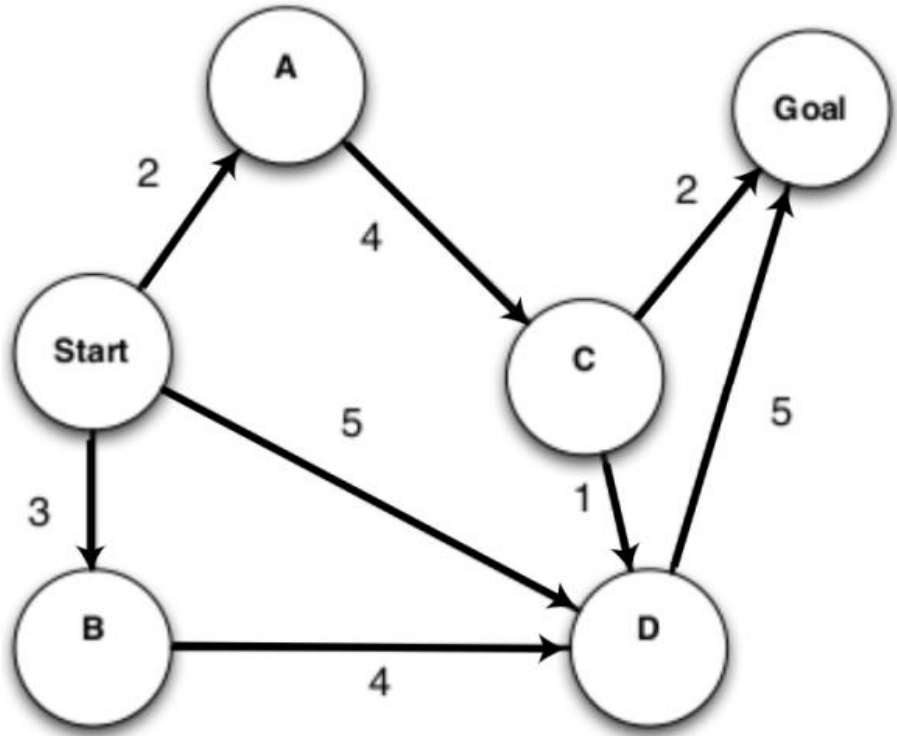
State, f-value
path-cost, Reached-pointer

- ✓ Denotes popped node
- ✗ Denotes node generated but discarded (not added to frontier)



Iter#	Popped
1	Start
2	A
3	C
4	Goal

Popped order: Start, A, C, Goal
DFS solution path: Start -> A -> C -> Goal
Solution cost: 8
Total nodes popped: 4
Total nodes generated: 7



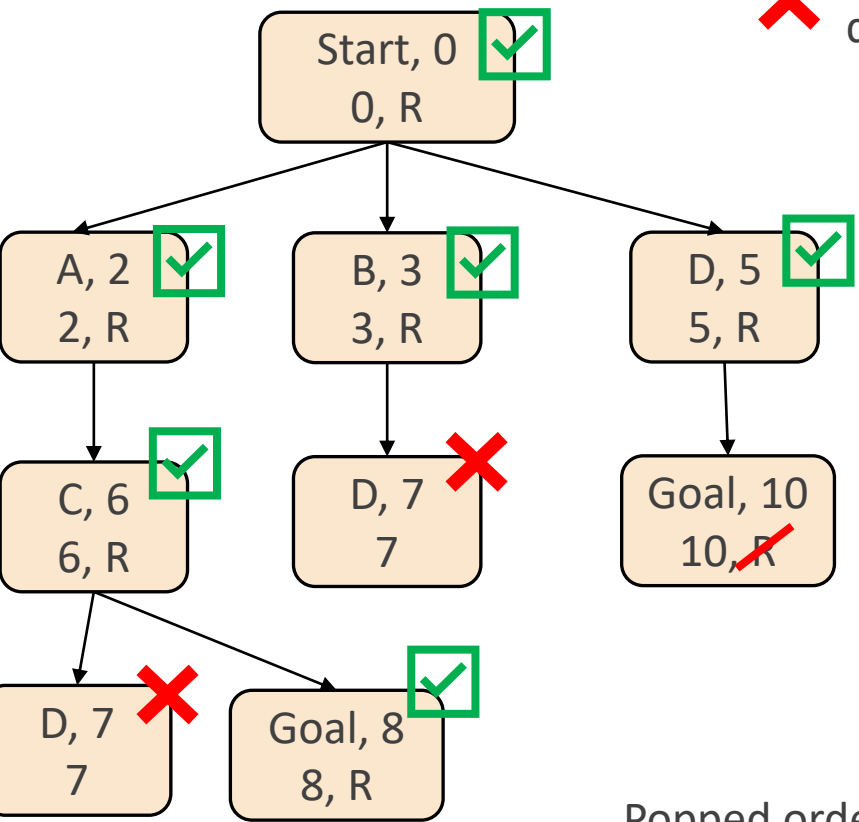
Note with C at iteration #3, D is not added to frontier.

Uninformed Search Practice – UCS

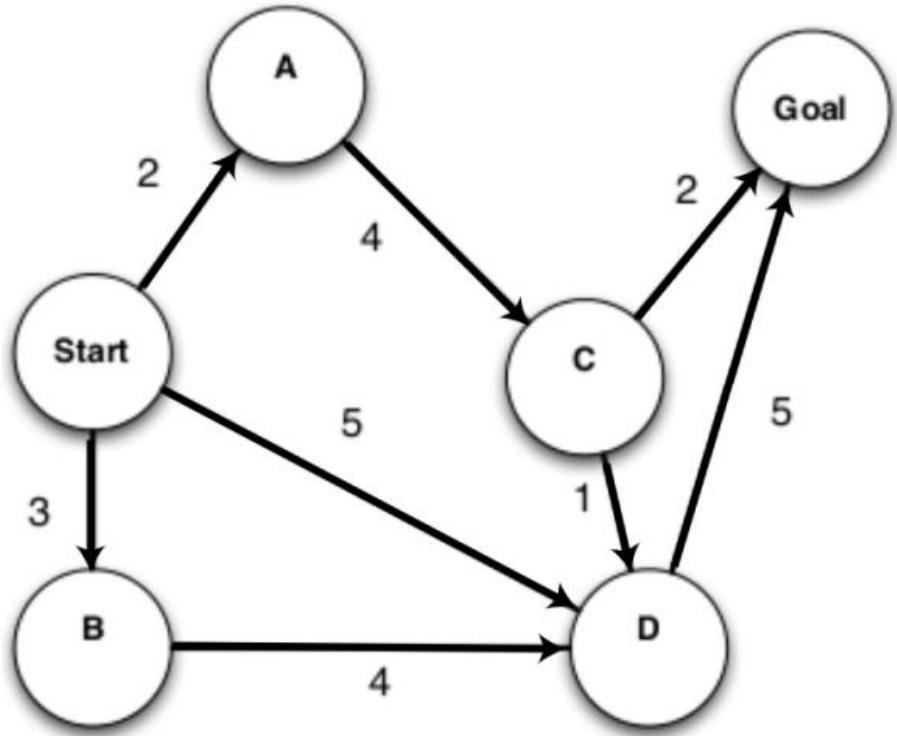
```
for each child in EXPAND(problem, node) do
  s ← child.STATE
  if s is not in reached or child.PATH-COST < reached[s].PATH-COST then
    reached[s] ← child
    add child to frontier
```

State, f-value
path-cost, Reached-pointer

✓ Denotes popped node
✗ Denotes node generated but discarded (not added to frontier)



Iter#	Popped
1	Start
2	A
3	B
4	D
5	C
6	Goal



Popped order: Start, A, B, D, C, Goal
UCS solution path: Start -> A -> C -> Goal
Solution cost: 8
Total nodes popped: 6 Total nodes generated: 9

Summary

Summary

- Define problem solving agents.
- Identify the components for the problem formulation.
- Be able to formulate the state of a search problem.
- Be able to implement the code for a search problem.
- Consider the task of searching for a solution in state space.
- Identify, describe, apply, evaluate different uninformed search strategies: BFS, DFS, DLS, IDS.
- Understand the difference between tree-like and graph-like search.
- Be able to implement searching algorithms.